Computers & Industrial Engineering 223 (2027) 112434

Contents lists available at ScienceDirect

Computers & Industrial Engineering

journal homepage: www.elsevier.com/locate/caie

Offline reinforcement learning with heterogeneous graph representation and attention mechanisms for human–machine collaborative flexible job shop scheduling

Shiliang Shao [a], Maolin Cai [a], Hongquan Qu [a], Qihui Yu [b], Xiaomeng Tong [a,*]

[a] School of Automation Science and Electrical Engineering, Beihang University, Beijing 100080, China
[b] School of Mechanical Engineering, Inner Mongolia University of Science and Technology, Baotou 014010, China

ARTICLE INFO

Dataset link: DFJSP-W Datasets for Human--Machine Collaborative Flexible Job Shop Scheduling (Original data)

Keywords:
Human-machine collaboration
Flexible job shop scheduling
Multi-attention mechanism
Offline reinforcement learning

ABSTRACT

Industry 5.0 places human-centered production at the core of manufacturing, making effective coordination between workers and machines increasingly important. This study addresses the Dual-Resource Flexible Job Shop Scheduling Problem with Worker constraints (DFJSP-W), which requires joint decisions on operation sequencing, machine assignment, and worker allocation. We propose Multi-Attention Dueling Quantile-Based Actor–Critic with Uncertainty Control (MA-DQUAC), an offline reinforcement learning method for DFJSP-W. A heterogeneous scheduling graph and multi-attention encoder represent operation–machine–worker dependencies, while quantile-output dispersion modulates the actor and critic updates for actions poorly represented in the offline dataset. Across instances of different scales, MA-DQUAC achieves lower makespans than dispatching rules, a genetic algorithm, and adapted online and offline reinforcement learning baselines. The learned policies transfer to larger unseen instances and remain relatively stable under reductions in offline training data. Scalability tests and an industrial case study further show rapid inference and high-quality feasible scheduling for worker–machine collaborative production.

1. Introduction

Manufacturing is shifting from the technology-driven Industry 4.0 paradigm toward the value-driven and human-centered Industry 5.0 paradigm (Xu et al., 2021). Under this paradigm, production systems increasingly adopt human–machine collaborative architectures (Destouet et al., 2023). Effective coordination of human labor and machine resources is therefore essential for improving manufacturing flexibility and robustness (Chang et al., 2025). The Job Shop Scheduling Problem (JSP) is a classical resource-allocation problem in manufacturing systems (Zhang et al., 2019). The Flexible Job Shop Scheduling Problem (FJSP) extends JSP by allowing operations to be processed on alternative machines (Xie et al., 2019). However, practical production also depends on workers, whose skills affect both scheduling feasibility and production efficiency (Dauzère-Pérès et al., 2024; Li et al., 2025). The Dual-Resource Flexible Job Shop Scheduling Problem with Worker constraints (DFJSP-W) therefore requires the joint optimization of operation sequencing, machine assignment, and worker allocation. This coupled decision structure substantially enlarges the search space and imposes greater demands on scheduling quality and response speed (Aribi et al., 2023).

Traditional solving methods for the FJSP can be categorized into exact methods, heuristics, and meta-heuristics (Dauzère-Pérès et al., 2024; Qu et al., 2026; Xie et al., 2019). Exact methods aim to obtain theoretically optimal solutions through rigorous mathematical modeling, such as mixed-integer linear programming (Liu et al., 2024), constraint programming (Lan & Berkhout, 2025), and logic-based Benders decomposition (Naderi & Roshanaei, 2022; Xiong & Liu, 2025). However, constrained by the NP-hard nature of the DFJSP-W, their computational time increases exponentially with problem scale, leading to a severe "curse of dimensionality" in large-scale industrial scenarios. In contrast, heuristic and meta-heuristic methods achieve a balance between computational efficiency and solution quality (Qu et al., 2026). Priority rule-based heuristics are computationally efficient and widely used for real-time scheduling (Sobeyko & Mönch, 2016; Thenarasu et al., 2022). Genetic Algorithms (GA) (Zhang et al., 2011), Tabu Search (Li & Gao, 2016), Simulated Annealing (Defersha et al., 2022), and Memetic Algorithms (Qu et al., 2024; S. Yuan et al., 2025) further optimize performance through global evolutionary mechanisms. Building on this, hyper-heuristic methods represented by Genetic Programming have been introduced to automate the evolution of scheduling heuristics.

* Corresponding author.
E-mail addresses: ssl020104@buaa.edu.cn (S. Shao), caimaolin@buaa.edu.cn (M. Cai), qhongquan@buaa.edu.cn (H. Qu), yuqihui@imust.edu.cn (Q. Yu), tongxiaomeng@buaa.edu.cn (X. Tong).

https://doi.org/10.1016/j.cie.2026.112434
Received 17 July 2026; Received in revised form 29 September 2026; Accepted 2 October 2026
Available online 10 October 2026
0360-8352/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

By learning Pareto sets (Xu et al., 2025), performing multi-objective optimization (Pang et al., 2025), or mining composite rules (Li et al., 2024), these methods significantly enhance the algorithms' adaptability to dynamic and uncertain environments. However, when applied to DFJSP-W, traditional methods still face serious challenges. Simple heuristics struggle to capture the complex coupling between dual resources, resulting in limited solution quality. Meta-heuristic algorithms and genetic-programming methods face repeated-search costs and limited representational capacity, respectively, reducing their suitability for real-time scheduling.

To overcome the limitations of traditional methods, data-driven deep reinforcement learning (DRL) has emerged as an end-to-end approach for dynamic and large-scale scheduling decisions (Zhang & Zhu, 2025). Zhang et al. (2020) introduced DRL to JSP by combining disjunctive graph modeling with a graph neural network (GNN), enabling learned policies to generalize across problem scales. Subsequent FJSP studies used a heterogeneous GNN to capture spatiotemporal relationships (Song et al., 2022) and a dual-attention network to model operation–machine interactions (Wang et al., 2023). DRL has also been extended to AGV collaborative scheduling (Yuan et al., 2025), large-scale system optimization (Moon et al., 2024), and hybrid evolutionary frameworks (J. Yang et al., 2025). For human–machine collaborative scheduling, An et al. (2025) modeled operator preferences through a multi-behavior graph convolutional network, whereas Zhong et al. (2025) used multi-domain graph attention to represent worker-skill heterogeneity. However, these online reinforcement learning (online RL) methods depend on repeated interactions with the environment, creating cost and safety barriers to deployment in physical production systems (Leng et al., 2022; Zhang & Zhu, 2025). Consequently, learning scheduling policies from fixed datasets without additional environment interaction has emerged as an important research direction.

Against this background, offline reinforcement learning (offline RL), which learns from static observational datasets, has attracted increasing attention as a practical approach for real-world applications (Prudencio et al., 2023). Its effective use of historical data has supported applications in healthcare (Ghasemi et al., 2025), environmental protection (R. Yang et al., 2025), finance (Bai et al., 2025), large language models (Wang et al., 2025), robotics (Su et al., 2025), and electric vehicles (Y. Wang et al., 2025). Despite this progress, distribution shift between the offline dataset and the learned policy can cause value overestimation for out-of-distribution (OOD) actions and even policy collapse (Prudencio et al., 2023). To address this challenge, a series of offline RL methods have been proposed to suppress extrapolation errors and enhance policy robustness. Fujimoto et al. (2019) proposed the BCQ algorithm, which constrains the action generation range and forces the agent to make decisions within the support of the dataset. Kumar et al. (2020) introduced Conservative Q-Learning (CQL), which incorporates a conservatism regularization term into the Bellman error to effectively suppress the values of OOD actions. Nair et al. (2020) employed implicit policy constraints to alleviate distributional shifts during pretraining with suboptimal data. Wu et al. (2021) introduced uncertainty quantification mechanisms that dynamically weight training samples to reduce the influence of unreliable data. Dabney et al. (2018) achieved an accurate characterization of value distributions, while Christodoulou (2019) improved policy stability and generalization performance through a maximum-entropy framework. In addition, trajectory optimization approaches that model offline RL as a sequence prediction problem (Janner et al., 2021), as well as policy evaluation methods that correct stationary distribution bias via density ratio estimation (R. Zhang et al., 2020), provide strong support for offline policy evaluation from different perspectives. In the field of shop scheduling, van Remmerden et al. (2025b) and van Remmerden et al. (2025a) pioneered the application of offline RL to scheduling problems. They proposed the Offline Learned Dispatching (Offline-LD) and Conservative Discrete Quantile Actor-Critic (CDQAC) algorithms for JSP and FJSP, respectively, demonstrating the feasibility of learning scheduling policies from historical data.

Despite recent progress in applying offline RL to conventional shop scheduling, worker-constrained settings remain underexplored. Applying offline RL to DFJSP-W presents two closely related challenges. First, incorporating worker allocation transforms each scheduling decision into a feasible operation–machine–worker triplet, requiring explicit representation of heterogeneous dependencies among the three entity types. Second, the expanded action space can leave many feasible triplets sparsely represented in the offline dataset, increasing the risk of unreliable value estimates and policy updates.

To address these challenges, we propose Multi-Attention Dueling Quantile-Based Actor–Critic with Uncertainty Control (MA-DQUAC) for learning worker-constrained scheduling policies from fixed datasets. Unlike graph-based online RL methods for worker-related scheduling, MA-DQUAC requires no additional environment interaction during training. To capture the operation–machine–worker coupling overlooked by existing offline scheduling methods, it represents each decision as a feasible triplet and encodes heterogeneous dependencies through an heterogeneous scheduling graph (HSG) and multi-attention encoder. It further uses quantile-output dispersion as an uncertainty proxy to weight policy and value updates for sparsely represented actions. Experimental results on the evaluated DFJSP-W instances show that MA-DQUAC achieves lower makespans than the considered Priority Dispatching Rules (PDRs), GA, and the adapted online and offline RL baselines. It also obtains lower makespans than the time-limited OR-Tools reference on larger instances, transfers to larger unseen instances, and remains relatively stable when the offline training data are reduced.

In summary, the main contributions of this paper are as follows:

(1) For DFJSP-W, a Markov decision process (MDP) model is established, together with an HSG representation of the coupling relationships among operations, machines, and workers.

(2) A multi-attention feature extraction architecture is designed to capture interactions among operations, machines, and workers.

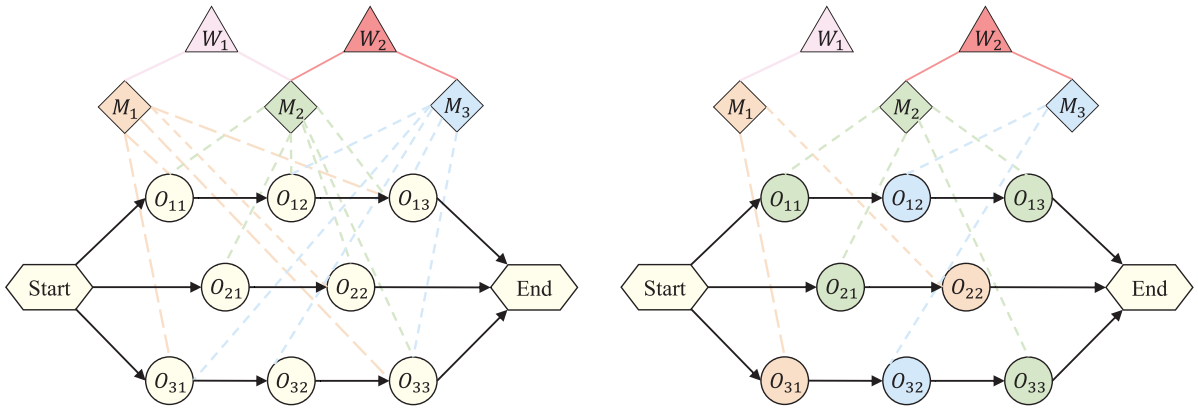
(3) An offline RL algorithm with uncertainty modulation is proposed. On the evaluated instances, MA-DQUAC achieves better solution quality than the considered traditional and online/offline RL baselines and can be applied to larger unseen instances.

The remainder of this paper is organized as follows: Section 2 elaborates on the necessary preliminaries. Section 3 introduces the proposed method in detail. Section 4 provides the experimental results and analysis. Section 5 summarizes the work and outlines future research directions.

2. Preliminaries

2.1. DFJSP-W and HSG

DFJSP-W is a significant extension of the classic FJSP. In a typical manufacturing unit, consider a set of jobs $\mathcal{J} = \{J_1, J_2, \ldots, J_n\}$, a set of machines $\mathcal{M} = \{M_1, M_2, \ldots, M_k\}$, and a set of workers $\mathcal{W} = \{W_1, W_2, \ldots, W_l\}$. Each job J_i $(i = 1, \ldots, n)$ contains a series of ordered operations O_{ij} $(j = 1, \ldots, n_i)$, and there are strict technological precedence constraints between operations. Unlike FJSP, DFJSP-W requires each operation O_{ij} to be assigned a compatible machine $M_k \in \mathcal{M}$ and a worker $W_l \in \mathcal{W}$ who performs the corresponding machine setup. These dual resource constraints increase the problem's complexity. The actual processing time p_{ijk} of operation O_{ij} is determined by machine performance, while workers incur specific setup times when preparing tasks on different machines, denoted as s_{ijkl}. The worker is released after completing the setup, whereas the machine remains occupied during both setup and processing. Therefore, the total machine occupancy time is $T_{ijkl} = s_{ijkl} + p_{ijk}$. The objective of the scheduling problem is to determine the setup start time S_{ij}, assigned machine M_k, and assigned



(a) DFJSP-W instance. (b) A possible solution.

Fig. 1. Heterogeneous scheduling graph for DFJSP-W.

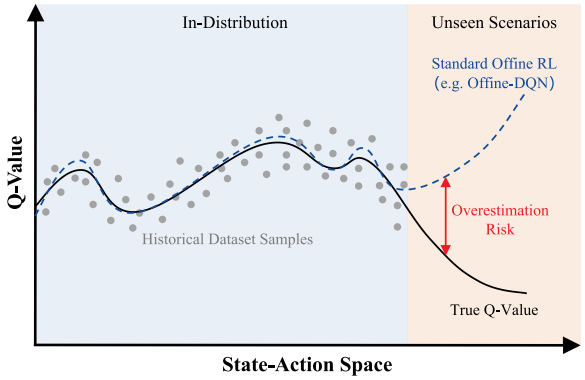


Fig. 2. Illustration of OOD overestimation risk in offline RL.

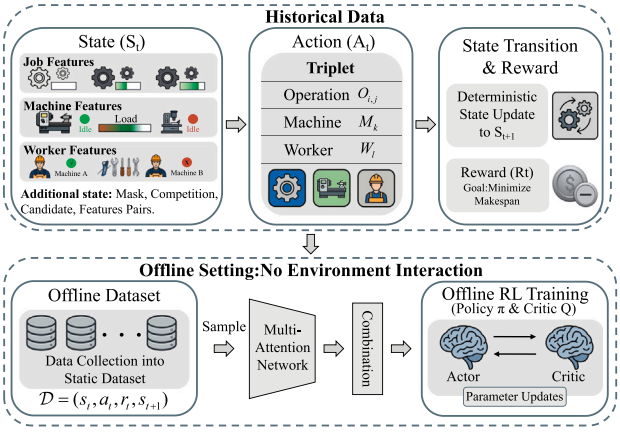


Fig. 3. Overview of the proposed method.

worker W_l for each operation to minimize the maximum completion time while satisfying all constraints, expressed as:

$$C_{\max} = \max_{i,j} C_{ij} \tag{1}$$

where C_{ij} is the completion time of the operation.

The main constraints of the problem include: (1) Operation precedence constraint: for the same job, the processing of a subsequent operation must start after its preceding operation is completed, although its setup may start earlier. (2) Machine and worker uniqueness constraint: at any time, each machine can be assigned to at most one operation, and each worker can perform at most one setup task. (3) Resource compatibility constraint: the assigned machine and worker must belong to their respective feasible sets. Detailed mathematical models are provided in Appendix A.

To effectively capture the complex, heterogeneous entity relationships and provide structured inputs to the subsequent multi-attention network, the DFJSP-W state is modeled as an HSG. Fig. 1 presents an example of a heterogeneous graph. Formally, the HSG is defined as $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. The node set $\mathcal{V} = \mathcal{V}_O \cup \mathcal{V}_M \cup \mathcal{V}_W$ contains operations, machines, and workers, while the edge set $\mathcal{E}$ comprises three types of connectivity: (1) Operation-operation edges: describing the technological workflow within a job; (2) Operation-machine edges: describing the processing feasibility of machines for operations; (3) Machine-worker edges: describing workers' eligibility to perform setups on machines. This graph structure not only explicitly encodes scheduling constraints but also provides rich state representations for the offline RL agent through node features.

2.2. OOD overestimation in offline RL

Although the HSG provides a structured representation of the scheduling state, offline policy learning remains constrained by the coverage of the fixed dataset. In DFJSP-W, the large operation–machine–worker action space leaves many feasible actions absent from or sparsely represented in the dataset, creating OOD risk. As illustrated in Fig. 2, Bellman backups involving these poorly supported actions can overestimate their Q-values. The resulting bias may propagate through subsequent updates and destabilize the learned policy. This propagation motivates the quantile-based uncertainty control mechanism introduced in Section 3.

3. Method

This section presents the main contributions of this work in detail, and an overview of the proposed method is illustrated in Fig. 3. Specifically, an MDP model for the DFJSP-W is first formulated to establish the mathematical benchmarks for states, actions, and rewards. Subsequently, a multi-attention feature extractor is designed to extract high-dimensional production state features via heterogeneous

graph representation learning. Finally, an offline RL training method is proposed to suppress value overestimation errors and enhance the generalization performance of the scheduling policy.

3.1. MDP formulation for DFJSP-W

In offline RL, the generation process of a scheduling solution is reflected as a sequence of decision trajectories stored in a static dataset. Specifically, at each decision step t, the action $a_t = (O_{ij}, M_k, W_l)$ corresponds to the state s_t, assigning operation O_{ij} to machine M_k and selecting worker W_l to perform the corresponding setup. This decision is accompanied by a state transition to s_{t+1} and a corresponding reward r_t; these transitions constitute the foundation of algorithmic learning. Learning the decision patterns contained in historical trajectories can support the development of an effective scheduling policy. Therefore, the MDP is defined as follows.

(1) **State:** At each decision step t, the system state s_t is represented as a set of five feature-tensor categories. First, job features ($H_O \in \mathbb{R}^{10}$), machine features ($H_M \in \mathbb{R}^8$), and operation-machine pair features ($H_{OM} \in \mathbb{R}^8$) follow the definitions of Wang et al. (2023). Second, tailored to the characteristics of the DFJSP-W, worker features ($H_W \in \mathbb{R}^6$) and machine-worker pair features ($H_{MW} \in \mathbb{R}^4$) are innovatively introduced. During the decision-making process, the operation-machine pair features H_{OM} and machine-worker pair features H_{MW} are further fused into H_{OMW}. This multi-level and multi-dimensional state representation provides rich static and dynamic attributes for scheduling decisions. Detailed state feature definitions are provided in Appendix B.

(2) **Action:** At each decision step, $\mathcal{O}_t^{\text{cand}}$ denotes the candidate operation set. It contains only the next unscheduled operation of each unfinished job, thereby enforcing the operation-precedence constraint before resource assignment. The candidate operation–machine–worker triplets are organized as the discrete action set

$$\widetilde{\mathcal{A}}(s_t) = \mathcal{O}_t^{\text{cand}} \times \mathcal{M} \times \mathcal{W}, \tag{2}$$

where each triplet $a = (O_{ij}, M_k, W_l)$ corresponds to one position in the unified discrete action representation. A dynamic feasibility mask $m(s_t, a) \in \{0, 1\}$ is defined over the same indexed triplets: $m(s_t, a) = 1$ if machine M_k can process operation O_{ij} and worker W_l can perform its setup on that machine; otherwise, $m(s_t, a) = 0$. Accordingly, the feasible action set is

$$\mathcal{A}(s_t) = \left\{ a \in \widetilde{\mathcal{A}}(s_t) \mid m(s_t, a) = 1 \right\}. \tag{3}$$

Thus, the mask maintains a one-to-one correspondence with the candidate triplets and dynamically filters infeasible resource assignments as the state evolves.

(3) **State Transition:** State transitions follow deterministic scheduling rules. Let A_i, A_k, and A_l denote the availability times of job J_i, machine M_k, and worker W_l, respectively. For action $a_t = (O_{ij}, M_k, W_l)$, the setup starts at $S_{ij} = \max\{A_k, A_l, A_i - s_{ijkl}\}$. Processing starts at $S_{ij} + s_{ijkl}$, and the operation is completed at $C_{ij} = S_{ij} + s_{ijkl} + p_{ijk}$. The worker is released at $S_{ij} + s_{ijkl}$, whereas the machine and job become available at C_{ij}.

(4) **Reward:** To minimize the makespan, a dense differential reward mechanism is employed. Let $C_{\text{est}}(s_t)$ denote an estimated completion-time indicator constructed from the current partial schedule. For each operation O_{ij}, we define its reference duration as

$$\widetilde{T}_{ij} = \min_{M_k \in \mathcal{M}_{ij}, W_l \in \mathcal{W}_{ijk}} \left(s_{ijkl} + p_{ijk} \right). \tag{4}$$

Let $\hat{C}_{ij}$ denote the completion-time estimate of O_{ij}. Initially, $\hat{C}_{ij}$ is obtained by accumulating $\widetilde{T}_{ij}$ along the precedence order of each job. After executing $a_t = (O_{ij}, M_k, W_l)$, $\hat{C}_{ij}$ is updated using the current availability times of the selected job, machine, and worker, and the resulting adjustment is propagated to the remaining operations of the same job. Thus, unfinished jobs remain included through their estimated completion times. Since $\hat{C}_{in_i}$ denotes the estimated completion time of job J_i, the state-level completion-time indicator is defined as

$$C_{\text{est}}(s_t) = \max_i \hat{C}_{in_i}. \tag{5}$$

The reward is then defined as

$$r_t = C_{\text{est}}(s_t) - C_{\text{est}}(s_{t+1}). \tag{6}$$

At the terminal state, C_{est} equals the realized makespan, and the differential reward provides dense feedback based on stepwise changes in the estimated completion time. The detailed computation procedure is provided in Appendix C.

(5) **Offline Setting:** Policy learning is based solely on a pre-collected static dataset, which consists of historical scheduling trajectories generated by various behavioral policies of varying quality. The training objective is to mitigate OOD action value overestimation caused by distribution shift using only $\mathcal{D}$, learn more reliable $Q_\theta(s, a)$ estimates, and derive an effective scheduling policy $\pi_\theta(a \mid s)$.

3.2. Multi-attention feature extractor

In the DFJSP-W scenario, scheduling decisions depend on the accurate modeling of complex coupling relationships among three types of heterogeneous entities encompassing operations, machines, and workers. Consequently, this paper constructs a Multi-Attention Feature Extractor (MAFE) based on the HSG to characterize information interaction structures across different resource dimensions in parallel, and its overall architecture is illustrated in Fig. 4.

MAFE extends the Dual Attention Network (DAN) proposed by Wang et al. (2023) by adding structural components. Within this framework, the Operation Message Attention Block and the Machine Message Attention Block are utilized to model the technological dependencies among operations within a job and the dynamic competitive relationships among machines regarding candidate operations, respectively. Both blocks inherit the original DAN design philosophy and attention computation scheme and are thus not elaborated further. To address the additional worker resource constraints in the DFJSP-W, this paper further introduces the Worker Message Attention Block (WAB). The WAB is used to model workers' competitive relationships regarding candidate operations and implicitly reflects their collaborative scheduling potential via an attention aggregation mechanism.

For each candidate operation $O_{ij} \in \mathcal{O}_t^{\text{cand}}$, let $b_{ij,u}(s_t) = 1$ if worker W_u can perform the setup for O_{ij} on at least one compatible machine, and $b_{ij,u}(s_t) = 0$ otherwise. For any two workers W_u and W_v, the competition feature vector at layer l is constructed as

$$\mathbf{v}_{uv}^{(l)}(s_t) = \sum_{O_{ij} \in \mathcal{O}_t^{\text{cand}}} b_{ij,u}(s_t) b_{ij,v}(s_t) \mathbf{h}_{O_{ij}}^{(l)}, \tag{7}$$

where $\mathbf{h}_{O_{ij}}^{(l)}$ is the feature of candidate operation O_{ij} at layer l. Accordingly, $\mathbf{v}_{uv}^{(l)}$ aggregates the scheduling status, workload, processing-time characteristics, and machine flexibility of the candidate operations for which both workers can perform setup. Based on the worker features and the competition feature, the attention coefficient is calculated as

$$z_{uv}^{(l)} = \text{LeakyReLU}\left(\mathbf{a}_W^\top \left[\mathbf{W}_W \mathbf{h}_u^{(l)} \,\|\, \mathbf{W}_W \mathbf{h}_v^{(l)} \,\|\, \mathbf{W}_V \mathbf{v}_{uv}^{(l)} \right] \right). \tag{8}$$

where $\mathbf{W}_W$, $\mathbf{W}_V$, and $\mathbf{a}_W^\top$ are learnable parameters. After softmax normalization and weighted aggregation, the worker embedding is obtained.

All the above attention modules adopt a multi-head attention mechanism to enhance representational capacity. After L layers of multi-attention propagation, the embeddings for operations ($h_{O_{ij}}^{(L)}$), machines ($h_{M_k}^{(L)}$), and workers ($h_{W_u}^{(L)}$) are respectively averaged via mean pooling and concatenated to form a global state representation:

$$h_G = \left(\frac{1}{|\mathcal{O}|} \sum_{O_{ij} \in \mathcal{O}} h_{O_{ij}}^{(L)} \right) \| \left(\frac{1}{|\mathcal{M}|} \sum_{M_k \in \mathcal{M}} h_{M_k}^{(L)} \right)$$

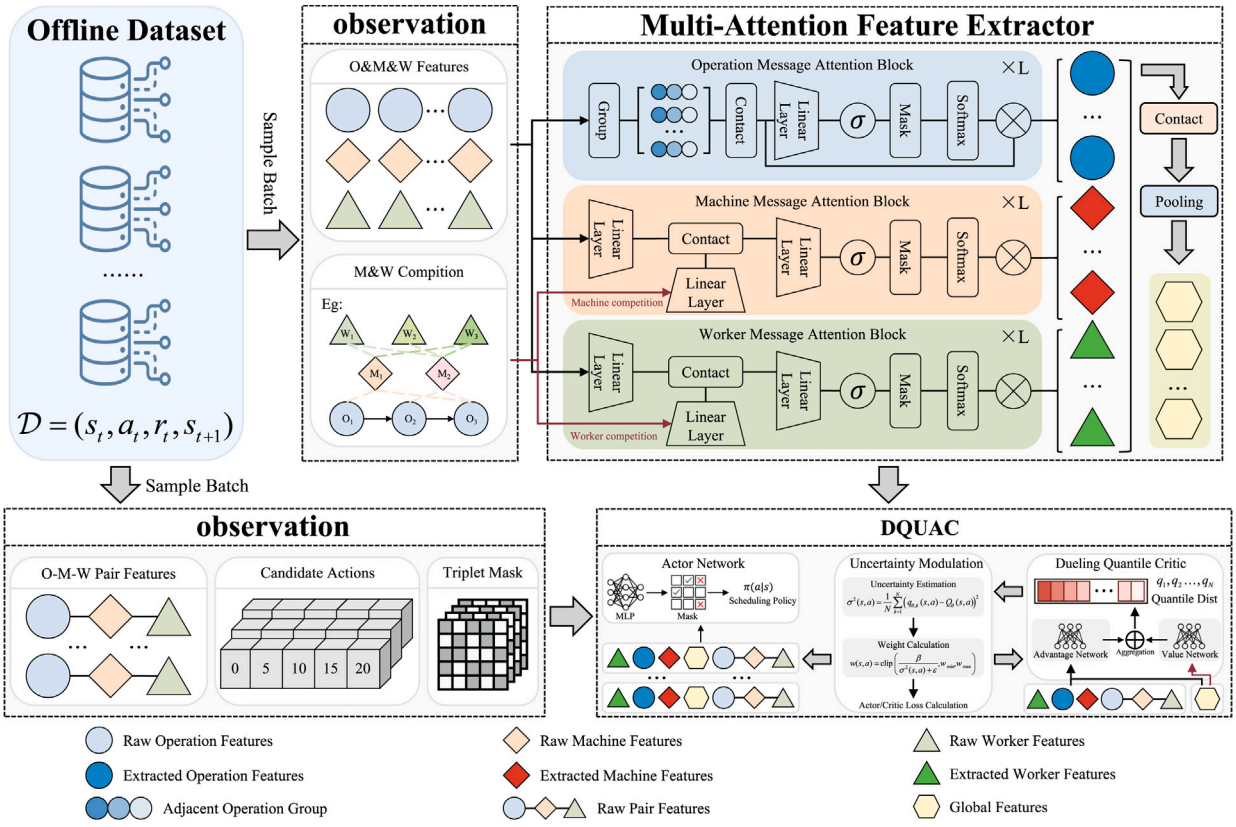


Fig. 4. Framework of MA-DQUAC.

$$\| \left(\frac{1}{|\mathcal{W}|} \sum_{W_u \in \mathcal{W}} h_{W_u}^{(L)} \right) \tag{9}$$

where $|\mathcal{O}|$, $|\mathcal{M}|$, and $|\mathcal{W}|$ denote the number of active operations, machines, and workers in the current state, respectively.

This embedding integrates global information on task urgency, machine load, and labor status, providing a compact state representation for the subsequent policy and value networks.

3.3. Dueling quantile-based actor-critic with uncertainty control

For the DFJSP-W scheduling problem in an offline environment, the main challenge is addressing the OOD evaluation bias introduced by the high-dimensional, discrete action space. To this end, this paper proposes a Dueling Quantile-Based Actor-Critic framework with Uncertainty Control (DQUAC), as illustrated in Fig. 4. The objective is to extract a scheduling policy with strong generalization capability from a limited static dataset. The specific components are described as follows.

3.3.1. Dueling quantile critic for distributional value estimation

Traditional value-based reinforcement learning methods typically estimate only the expected return and thus struggle to characterize stochastic risk in scheduling. Inspired by van Remmerden et al. (2025a), DQUAC adopts a distributional reinforcement learning framework (Dabney et al., 2018). It uses a set of discrete quantiles $\{q_k(s,a)\}_{k=1}^{N}$ to approximate the return distribution, where N denotes the number of quantiles. Under this framework, the action value can be computed as the mean of all quantiles:

$$Q(s,a) = \frac{1}{N} \sum_{k=1}^{N} q_k(s,a) \tag{10}$$

To mitigate overestimation bias in offline learning, this paper maintains two evaluator networks with independent parameters, denoted as θ_1 and θ_2, in parallel. For simplicity of expression, θ is used in subsequent formulas to represent the critic network parameters θ_i ($i \in \{1,2\}$). The network adopts the dueling architecture (Wang et al., 2016) to decouple representations. The state value stream $V(s)$ receives the global state embedding h_G extracted by MAFE to evaluate the baseline value distribution of the current global state. The action advantage stream $A(s,a)$ receives the local features $h_{\text{local}}(h_{O_{ij}}, h_{M_k}, h_{W_l}, H_{OMW})$ together with the global feature h_G, to capture the advantage distribution of a specific action relative to the average level. The two streams are aggregated via mean-centering to obtain the kth quantile output:

$$\begin{aligned} q_{\theta,k}(s,a) = & V_{\theta,k}(h_G) + \Big(A_{\theta,k}(h_{\text{local}}, h_G) \\ & - \frac{1}{|\mathcal{A}(s)|} \sum_{a' \in \mathcal{A}(s)} A_{\theta,k}(h'_{\text{local}}, h_G) \Big) \end{aligned} \tag{11}$$

During model training, updates are performed using the distributional Bellman operator. For a transition tuple (s,a,r,s') sampled from the dataset D, the jth quantile y_j of the target return distribution is defined as:

$$y_j = r + \gamma \min_{m=1,2} q_{\bar{\theta}_m,j}(s', a') \tag{12}$$

where γ is the discount factor, $\bar{\theta}_m$ represents the parameters of the target networks, and a' is sampled from the masked current policy π over the feasible action set $\mathcal{A}(s')$ in state s'. The mask removes physically infeasible scheduling actions, while the OOD risk of feasible but weakly supported actions is further controlled by the clipped double-critic target in Eq. (12), together with the conservative regularization and quantile-based uncertainty weighting introduced below.

For an individual transition (s,a,r,s'), the transition-level Temporal-Difference loss of the critic network employs the quantile Huber loss (Dabney et al., 2018) to calculate the discrepancy between the current distribution and the target distribution:

$$\ell_{\text{TD}}(\theta; s,a,r,s') = \frac{1}{N} \sum_{k=1}^{N} \sum_{j=1}^{N} \rho_{\tau_k}^{\kappa} \left(y_j - q_{\theta,k}(s,a) \right) \tag{13}$$

where $\rho^{\kappa}_{\tau_k}(\cdot)$ denotes the asymmetric Huber loss function.

Building on this, a CQL regularization term (Kumar et al., 2020) is introduced. By suppressing the Q-values of OOD actions and elevating the values of real actions in the dataset, the policy's robustness is further enhanced. The final critic loss function is defined as:

$$\mathcal{L}_{\text{Critic}}(\theta) = \mathbb{E}_{\mathcal{D}}\left[\ell_{\text{TD}}(\theta; s, a, r, s')\right] + \alpha_{\text{CQL}}\,\mathbb{E}_{\mathcal{D}}\left[\log \sum_{a' \in \mathcal{A}(s)} e^{Q_\theta(s,a')} - Q_\theta(s,a)\right]. \tag{14}$$

where both expectations are taken over transitions sampled from $\mathcal{D}$, and α_{CQL} is the regularization coefficient that controls the degree of conservatism. The second term in Eq. (14) is denoted by $\mathcal{L}_{\text{CQL}}(\theta)$ below.

This mechanism encourages the value network to characterize return distributions while producing more conservative estimates for OOD actions.

3.3.2. Actor network

The actor network takes (h_{local}, h_G) as input to learn the action policy $\pi_\psi(a \mid s)$. For each candidate triplet $a \in \widetilde{\mathcal{A}}(s)$, the corresponding operation, machine, and worker embeddings, together with the pair features in H_{OMW} and the global embedding h_G, are combined to generate a policy score $z_\psi(s,a)$. These entity embeddings are obtained by the separate attention modules described in Section 3.2; the actor does not construct pairwise attention among the candidate triplets. Instead, the scores for all positions in $\widetilde{\mathcal{A}}(s)$ are evaluated in one batched forward pass. The feasibility mask $m(s,a) \in \{0,1\}$ defined in Section 3.1 is then applied before policy normalization:

$$\pi_\psi(a \mid s) = \frac{\exp\big(z_\psi(s,a)\big)\, m(s,a)}{\sum_{a' \in \widetilde{\mathcal{A}}(s)} \exp\big(z_\psi(s,a')\big)\, m(s,a')}, \quad a \in \widetilde{\mathcal{A}}(s). \tag{15}$$

Because infeasible triplets have zero probability, the support of $\pi_\psi(\cdot \mid s)$ is the feasible action set $\mathcal{A}(s)$. For $s = s_t$, the number of policy scores is $|\widetilde{\mathcal{A}}(s_t)| = |\mathcal{O}_t^{\text{cand}}\|\mathcal{M}\|\mathcal{W}|$; thus, for fixed machine and worker sets, the current-step policy output grows linearly with the number of candidate operations rather than with the number of possible complete schedules.

The constraints are embedded in the policy gradient optimization process, allowing the policy to naturally follow scheduling-related physical constraints. The policy update loss function follows the same structure as that in soft actor-critic (Christodoulou, 2019):

$$\mathcal{L}_\pi(\psi) = \mathbb{E}_{s\sim\mathcal{D}}\left[\mathbb{E}_{a\sim\pi_\psi(\cdot|s)}\left[\alpha \log \pi_\psi(a \mid s) - Q_{\min}(s,a)\right]\right] \tag{16}$$

where α is the temperature coefficient, and $Q_{\min}(s,a)$ is the minimum of the quantile estimates output by the aforementioned twin critic networks.

3.3.3. Quantile-based uncertainty modulation

Although CQL suppresses optimistic estimation of OOD actions at the global level, it cannot characterize differences in uncertainty across individual samples. To further improve the granularity of offline learning, the method proposed by Wu et al. (2021) estimates predictive variance via Monte Carlo (MC) Dropout and weights the Bellman error by the inverse variance, thereby reducing the impact of highly uncertain targets during training. However, MC Dropout requires multiple forward passes, which increases computational cost and complicates its integration with quantile-based critic architectures. Motivated by this, a lightweight quantile-based uncertainty modulation mechanism is designed. Since the quantile critic directly outputs multiple quantiles of the return distribution, their dispersion provides an empirical proxy for action-level OOD uncertainty. For any state–action pair (s,a), the variance of its value distribution is calculated using the quantile set to construct an uncertainty weight coefficient:

$$\sigma^2(s,a) = \frac{1}{N}\sum_{k=1}^{N}\left(q_{\theta,k}(s,a) - Q_\theta(s,a)\right)^2 \tag{17}$$

$$w(s,a) = \text{clip}\left(\frac{\beta}{\sigma^2(s,a)+\epsilon},\ w_{\min},\ w_{\max}\right) \tag{18}$$

where β controls the intensity of uncertainty modulation, ϵ ensures numerical stability, and $[w_{\min}, w_{\max}]$ are used to avoid extreme sample weights.

During the critic update, the uncertainty weight is applied to the transition-level quantile Huber loss before averaging over the dataset, thereby reducing the interference of high-uncertainty samples on value function learning. The modified critic loss function is:

$$\mathcal{L}^{\text{DQUAC}}_{\text{Critic}}(\theta) = \mathbb{E}_{\mathcal{D}}\Big[w(s,a)\,\ell_{\text{TD}}(\theta; s, a, r, s')\Big] + \mathcal{L}_{\text{CQL}}(\theta). \tag{19}$$

Thus, each transition is weighted according to its estimated uncertainty before the minibatch loss is averaged.

During actor policy improvement, the weight adjusts the policy gradient to emphasize actions with more confident value estimates and reduce the influence of high-variance estimates. The weighted actor objective function is:

$$\mathcal{L}^{\text{DQUAC}}_{\pi}(\psi) = \mathbb{E}_{s\sim\mathcal{D}}\left[\sum_a \pi_\psi(a \mid s) w(s,a) \cdot \big(\alpha \log \pi_\psi(a \mid s) - Q_{\min}(s,a)\big)\right] \tag{20}$$

Compared with the original method, this mechanism has a lightweight implementation, as it requires neither additional Dropout layers nor multiple sampling. Moreover, because the variance is computed directly from the learned return distribution, it provides an interpretable signal for uncertainty modulation in offline DFJSP-W scheduling.

4. Experiment

4.1. Problem instance generation and offline dataset construction

To evaluate the performance and generalization characteristics of the proposed algorithm in the DFJSP-W scenario, this paper constructs a set of multi-scale problem instances based on the classic FJSP benchmark generation protocol (Brandimarte, 1993), while further incorporating worker-related resource constraints. Existing JSP and FJSP benchmarks mainly provide problem instances, but they usually do not include worker-dependent setup times or the operation-machine-worker coupling structure required by DFJSP-W. Therefore, the instance generation adopts a stratified random sampling strategy. First, a set of compatible machines is randomly generated for each operation, with processing times following a uniform distribution $U[1,50]$. Subsequently, compatible workers are generated for the selected machines, and the corresponding worker-dependent setup times also follow $U[1,50]$. This process forms an operation–machine–worker setup-time tensor. The experimental design includes five instance scales (denoted as $n \times m \times w$, representing the number of jobs, machines, and workers, respectively): $10 \times 5 \times 3$, $15 \times 10 \times 5$, $20 \times 10 \times 5$, $30 \times 10 \times 8$, and $40 \times 10 \times 8$. For each of the three training scales ($10 \times 5 \times 3$, $15 \times 10 \times 5$, and $20 \times 10 \times 5$), 100 instances are independently generated for training and another 100 instances for testing. The $30 \times 10 \times 8$ and $40 \times 10 \times 8$ scales are reserved exclusively as unseen generalization test sets, with 100 instances generated at each scale.

It should be noted that, in this study, problem instances and the static datasets used for offline RL are two different concepts. Problem instances define the scheduling environment, whereas static datasets consist of state–action-reward-next-state trajectories collected on the training instances. Since standard JSP/FJSP benchmarks do not provide such transition trajectories, this study constructs five types of independent static datasets using different behavior policies, covering various distribution patterns from high-quality expert demonstrations

Table 1
Hyperparameter tuning and fixed settings.

Hyperparameter	Candidate range	Final value
Tuned hyperparameters		
Actor learning rate	$\{3\times10^{-3}, 3\times10^{-4}, 3\times10^{-5}\}$	3×10^{-5}
Critic learning rate	$\{3\times10^{-3}, 3\times10^{-4}, 3\times10^{-5}\}$	3×10^{-4}
CQL coefficient α_{CQL}	$\{0.1, 0.5, 1.0, 2.0\}$	1.0
Temperature α	$\{1\times10^{-4}, 1\times10^{-3}, 1\times10^{-2}\}$	1×10^{-3}
Uncertainty scaling β	$\{0.5, 1.0, 1.5, 2.0\}$	1.5
Weight clip range	$\{[0.1, 2.0], [0.1, 5.0], [0.01, 10.0]\}$	$[0.1, 5.0]$
Fixed model and training settings		
Embedding dimension (D)	32	
Attention layers (L)	2	
MLP structure	2 layers, 64 hidden	
Dropout (p)	0.2	
Optimizer	Adam	
Batch size	128	
Training steps	100,000	
Quantiles (N)	32	
Discount factor (γ)	0.99	
Soft update (τ)	0.005	
Random seeds	0, 1, 2, 3, 4	

to random interactions. Specifically: (1) Expert: high-quality trajectories generated by OR-Tools within an 1800-second time limit, used to evaluate the algorithm's ability to learn from high-quality but narrow-distribution demonstrations. (2) **Expert-Noisy:** a mixture of clean and noise-perturbed expert trajectories. For each problem instance, a noise-perturbed trajectory was generated with a probability of 0.5; otherwise, the original expert trajectory was retained. Within a noise-perturbed trajectory, at each decision step, the expert action was replaced, with a probability of 0.5, by an action randomly selected from the current feasible action set $\mathcal{A}(s_t)$, with all feasible actions having an equal selection probability. With the remaining probability of 0.5, the expert action was executed if it remained feasible. If the expert action became infeasible because of an earlier random decision, a feasible action was randomly selected instead. This action-level stochasticity expands the state–action coverage while preserving a substantial proportion of high-quality expert decisions. (3) Mix-PDRs: mixed trajectories from four worker-adapted PDRs (FIFO, MOPNR, SPT, and MWKR), representing sub-optimal policies based on local greedy decisions. (4) GA: high-quality trajectories generated by a genetic algorithm. (5) Random: trajectories generated by uniformly sampling actions within the feasible action space; although the average return is low, it provides the broadest state–action coverage.

4.2. Experimental settings

The model is implemented in PyTorch. To keep the tuning process manageable and consistent with the reference architecture, the main architectural settings of the multi-attention feature extractor are fixed following Wang et al. (2023). The remaining training-related hyperparameters are tuned using a limited sequential grid search on the $10 \times 5 \times 3$ Expert-Noisy validation setting. Specifically, one hyperparameter is varied at a time, while the other settings are kept fixed to the default values or to the best values identified in previous tuning steps. This avoids the excessive cost of exhaustively enumerating all Cartesian combinations while still examining the key factors affecting offline RL stability. The final value of each tuned hyperparameter is selected according to the validation makespan and training stability. The candidate ranges of the tuned hyperparameters and the fixed experimental settings are summarized in Table 1. Additional validation curves for the tuned hyperparameters are provided in Appendix D.

All experiments were conducted on a server equipped with an Intel Xeon Platinum 8370C CPU (32 cores) and a single NVIDIA RTX A6000 GPU (48 GB VRAM).

4.3. Baseline and performance metrics

Four categories of representative methods are selected as comparison baselines. OR-Tools, GA, and PDRs serve different roles here and in offline dataset generation. In Section 4.1, they are used as behavior policies to collect trajectories from the training instances, whereas here they are independently evaluated on the held-out test instances. Therefore, no test-instance trajectories are used during offline training, avoiding data leakage and circular comparison. Specifically, (1) PDRs: four scheduling rules with strong performance in related studies are adopted, namely FIFO, MOPNR, SPT, and MWKR (Sels et al., 2012). (2) OR-Tools: OR-Tools is used as a time-limited constraint-programming reference solver, with a maximum solution time of 1800 s. If optimality is not proven within this limit, the best feasible solution is retained, and the reported gap is calculated as $\mathrm{Gap}_i = 100(C_i - \bar{C}_{\mathrm{OR}})/\bar{C}_{\mathrm{OR}}$, where C_i is the method's makespan on instance i and $\bar{C}_{\mathrm{OR}}$ is the mean OR-Tools makespan across the 100 test instances at that scale. (3) GA: a standard GA is adopted as a representative traditional search method for evaluating global evolutionary search under complex constraints (Chen et al., 1999). (4) RL methods: the selected RL baselines are the offline RL method CDQAC (van Remmerden et al., 2025a), the online RL method DANIEL (Wang et al., 2023), and the graph-based online DRL method PER-AC-PPO (Hu et al., 2025).

CDQAC was originally developed for JSP and FJSP, DANIEL for FJSP without worker constraints, and PER-AC-PPO for a multi-objective DRC-DFJSP with worker-related factors. CDQAC and DANIEL are adapted to DFJSP-W by incorporating worker representations and triplet decisions. PER-AC-PPO is evaluated using its independently trained, adapted makespan-oriented configuration at the original endpoint
$(\omega_1, \omega_2) = (1, 0)$. Table 2 summarizes the task-specific adaptations and the mechanisms retained from each original method.

Two inference strategies, greedy and sampling, are evaluated (Song et al., 2022). The greedy strategy selects the action with the highest output probability, reflecting real-time response capability, while sampling performs 100 parallel action samplings for each instance and records the best solution to explore higher-quality schedules with limited additional computational overhead. Detailed implementation, task-specific adaptation, training settings, random-seed controls, and fairness protocols for all baselines are provided in Appendix E. In Tables 3, 4, 5, and 7, Objective and Gap are first averaged over the same 100 test instances for each seed, then summarized as mean ± sample standard deviation across five independent training runs (independent solver runs for GA). Deterministic PDR results and Time are reported as means only.

4.4. Overall performance comparison

First, the training performance of MA-DQUAC was evaluated on five static datasets with different compositions, and the experimental results are reported in Table 3. MA-DQUAC achieves the best performance on the Expert-Noisy dataset. This indicates that, in the DFJSP-W scenario, moderate behavioral noise improves the effectiveness of policy learning. This phenomenon differs from the reliance on narrowly distributed expert trajectories in the robotics control domain Zhou et al. (2022). Also, it differs from the conclusion of van Remmerden et al. (2025a) that "purely random data can generalize effectively", because in the high-dimensional human-machine collaborative scheduling space, the model requires high-quality examples to guide the search direction, while also requiring sufficient randomness to improve state–action coverage. Therefore, the superior performance on the Expert-Noisy dataset suggests that both data quality and behavioral diversity are important for offline RL-based scheduling.

In addition, the average performance of MA-DQUAC across all datasets is superior to that of the offline RL baseline CDQAC, and MA-DQUAC also achieves a slightly lower average inference time.

Table 2
Adaptation protocol for learning-based baselines under DFJSP-W.

Baseline	Original formulation	DFJSP-W adaptation	Preserved mechanisms
CDQAC	offline RL for JSP and FJSP with dual-attention encoding and conservative quantile Actor–Critic learning.	Retains the operation–machine representation, encodes workers with a shared MLP, extends actions to feasible operation–machine–worker triplets, and uses the same offline datasets as MA-DQUAC.	Dual-attention encoder, twin dueling quantile critics, CQL regularization, and delayed policy updates.
DANIEL	Operation–machine pair decisions for FJSP with dual-attention encoding.	Encodes worker features with a shared MLP and extends direct action scoring to feasible operation–machine–worker triplets.	Operation and machine attention, machine-competition modeling, Actor–Critic PPO, and greedy/sampling decoding.
PER-AC-PPO $(1,0)$	Multi-objective DRC-DFJSP with a heterogeneous operation–machine–worker graph and weight-specific policies.	Maps the DFJSP-W state to the original graph without adding fatigue or cost features. Retrains the predefined makespan endpoint $(1,0)$ and applies the common reward and feasibility mask.	novel heterogeneous graph neural network (NHGNN), triplet decision, Actor–Critic PPO, prioritized experience replay, and adaptive clipping.

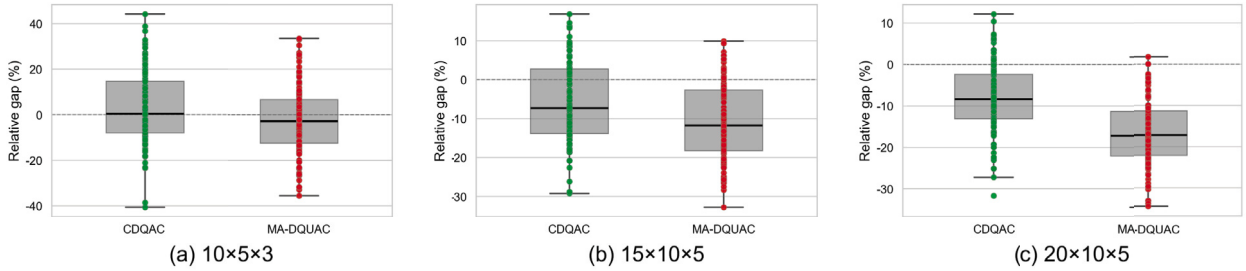


Fig. 5. Result distributions of CDQAC and MA-DQUAC.

Table 3
Offline RL results on different training datasets and instance scales.

Scale	Strategy	Training dataset	CDQAC			MA-DQUAC			OR-Tools
			Objective	Gap	Time (s)	Objective	Gap	Time (s)	
$10\times5\times3$	Greedy	Mix-PDRs	499.58 ± 3.18	48.22 ± 0.94%	0.6991	**476.80** ± 4.07	**41.46** ± 1.21%	0.6930	337.06
		GA	468.99 ± 3.10	39.14 ± 0.92%	0.7115	**461.58** ± 3.47	**36.94** ± 1.03%	0.7029	
		Expert	399.68 ± 2.66	18.58 ± 0.79%	0.7016	**390.10** ± 2.43	**15.74** ± 0.72%	0.6924	
		Expert-Noisy	384.17 ± 2.64	13.98 ± 0.78%	0.7042	**377.60** ± 2.76	**12.03** ± 0.82%	0.6890	
		Random	703.19 ± 3.41	108.62 ± 1.01%	0.7190	**519.27** ± 4.72	**54.06** ± 1.40%	0.6927	
	Sampling	Mix-PDRs	432.79 ± 1.75	28.40 ± 0.52%	1.4295	**376.28** ± 2.67	**11.64** ± 0.79%	1.4092	
		GA	**382.09** ± 2.50	**13.36** ± 0.74%	1.4638	385.96 ± 2.70	14.51 ± 0.80%	1.4551	
		Expert	354.31 ± 2.00	5.12 ± 0.59%	1.4174	**331.78** ± 2.00	**−1.57** ± 0.59%	1.4043	
		Expert-Noisy	347.28 ± 2.21	3.03 ± 0.66%	1.4277	**327.99** ± 2.19	**−2.69** ± 0.65%	1.4050	
		Random	574.93 ± 2.38	70.57 ± 0.71%	1.4109	**425.62** ± 3.28	**26.27** ± 0.97%	1.3890	
$15\times10\times5$	Greedy	Mix-PDRs	335.00 ± 2.64	48.22 ± 1.17%	1.0725	**306.15** ± 1.94	**35.46** ± 0.86%	1.0523	226.01
		GA	408.29 ± 2.95	80.65 ± 1.31%	1.0894	**369.17** ± 2.87	**63.34** ± 1.27%	1.0992	
		Expert	261.37 ± 1.31	15.65 ± 0.58%	1.0759	**242.22** ± 1.28	**7.17** ± 0.57%	1.0559	
		Expert-Noisy	237.12 ± 1.18	4.92 ± 0.52%	1.0778	**234.14** ± 1.13	**3.60** ± 0.50%	1.0553	
		Random	647.56 ± 6.13	186.52 ± 2.71%	1.0667	**473.52** ± 4.67	**109.51** ± 2.07%	1.0596	
	Sampling	Mix-PDRs	310.29 ± 1.90	37.29 ± 0.84%	4.1713	**244.89** ± 1.28	**8.35** ± 0.56%	4.0793	
		GA	315.13 ± 1.85	39.43 ± 0.82%	4.4136	**300.17** ± 2.00	**32.81** ± 0.89%	4.2969	
		Expert	225.31 ± 1.05	−0.31 ± 0.47%	4.1938	**206.00** ± 0.84	**−8.85** ± 0.37%	4.0937	
		Expert-Noisy	213.14 ± 0.94	−5.69 ± 0.41%	4.1697	**201.22** ± 0.91	**−10.97** ± 0.40%	4.0956	
		Random	544.47 ± 4.17	140.91 ± 1.84%	4.1184	**379.24** ± 2.51	**67.80** ± 1.11%	4.1222	
$20\times10\times5$	Greedy	Mix-PDRs	395.64 ± 2.79	36.50 ± 0.96%	1.4688	**359.44** ± 2.07	**24.01** ± 0.71%	1.4195	289.85
		GA	482.97 ± 3.68	66.63 ± 1.27%	1.4919	**401.07** ± 2.63	**38.37** ± 0.91%	1.4559	
		Expert	305.85 ± 1.74	5.52 ± 0.60%	1.4070	**293.14** ± 1.43	**1.14** ± 0.49%	1.4110	
		Expert-Noisy	298.53 ± 1.24	2.99 ± 0.43%	1.4366	**275.27** ± 1.16	**−5.03** ± 0.40%	1.4230	
		Random	799.93 ± 9.51	175.98 ± 3.28%	1.4394	**521.77** ± 4.89	**80.01** ± 1.69%	1.4135	
	Sampling	Mix-PDRs	344.35 ± 1.94	18.80 ± 0.67%	6.6161	**302.02** ± 1.32	**4.20** ± 0.45%	6.7855	
		GA	393.60 ± 2.25	35.79 ± 0.78%	6.7783	**339.18** ± 1.95	**17.02** ± 0.67%	6.9435	
		Expert	284.86 ± 1.35	−1.72 ± 0.47%	6.7619	**251.16** ± 1.07	**−13.35** ± 0.37%	6.3893	
		Expert-Noisy	265.85 ± 1.00	−8.28 ± 0.34%	6.7993	**241.56** ± 0.96	**−16.66** ± 0.33%	6.4337	
		Random	610.00 ± 5.42	110.45 ± 1.87%	6.9180	**423.53** ± 2.75	**46.12** ± 0.95%	6.6872	
	Average		407.54	44.64	2.6084	**347.93**	**23.08%**	2.5635	284.31

Note: Boldface indicates the best result between the two methods (columns) for each training dataset (row).

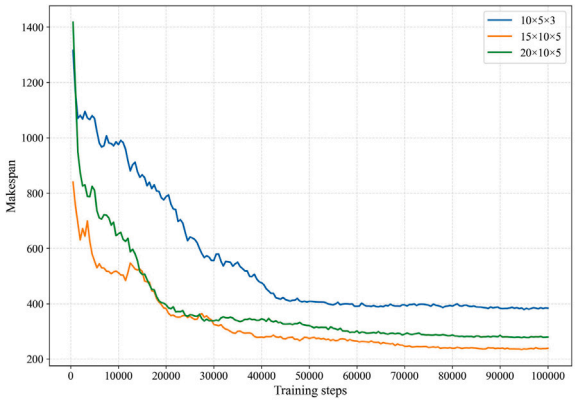


Fig. 6. Training curves under different scales.

For example, on the 20 × 10 × 5 Random dataset, CDQAC yields a sampling gap of 110.45%, which is far worse than the 46.12% achieved by MA-DQUAC. Even under the best-performing Expert-Noisy setting, MA-DQUAC (−16.66%) still outperforms CDQAC (−8.28%). Paired comparisons on the same test instances showed significantly lower Objective values for MA-DQUAC in 24 of the 30 settings (Holm-adjusted $p < 0.05$). The mean differences and 95% confidence intervals are reported in Appendix F. Furthermore, Fig. 5 presents the distribution of results for the two algorithms on the test instances. MA-DQUAC produces a more concentrated result distribution and a lower median, whereas CDQAC shows several high-gap outliers on the 20 × 10 × 5 instances. In comparison, MA-DQUAC generally achieves lower mean gaps, with smaller standard deviations in most evaluated conditions. In Table 3, OR-Tools is used as a time-limited reference solver, while MA-DQUAC performs fast amortized inference using a fixed policy after offline training. Therefore, it is reasonable that OR-Tools performs better in most settings, especially on small- and medium-scale instances such as 10 × 5 × 3 and 15 × 10 × 5. Similar observations have also been reported in related studies (Song et al., 2022; van Remmerden et al., 2025a, 2025b; Wang et al., 2023; Zhong et al., 2025). The main advantage of MA-DQUAC lies in learning reusable scheduling knowledge from static datasets and rapidly generating feasible schedules for repeated or large-scale scheduling scenarios.

Based on the conclusions in Table 3, the MA-DQUAC model trained on the Expert-Noisy dataset is selected as the final version. Fig. 6 shows the training curves at three different scales, depicting the trend in the average makespan across 100 validation instances over training epochs. It can be observed that MA-DQUAC exhibits smooth, stable convergence across all test scales.

Table 4 presents the performance metrics of the compared algorithms. Experimental results show that MA-DQUAC obtains lower gaps than the evaluated baselines, with larger differences on the larger tested instances. Taking the 20 × 10 × 5 large-scale instances as an example, the GA and the best-performing rules among the PDRs struggle to provide high-quality solutions. In contrast, MA-DQUAC achieves gap values of −5.03% and −16.66% using Greedy and Sampling strategies, respectively, showing competitive performance against the time-limited OR-Tools reference. Compared with the graph-based online RL baseline DANIEL, MA-DQUAC achieves substantially smaller gaps across all instance scales, indicating that directly extending a machine-oriented FJSP policy to worker-constrained scenarios is insufficient. Compared with the adapted makespan-oriented configuration of PER-AC-PPO at $(1, 0)$, MA-DQUAC achieves better solution quality under both inference strategies. For example, on the 20 × 10 × 5 instances, this configuration obtains gaps of 1.49% and −5.14% under Greedy and Sampling, respectively, whereas MA-DQUAC further reduces them to −5.03% and −16.66%. Across Table 4 conditions, MA-DQUAC achieved lower Objective values in all 42 paired comparisons with the baselines (Holm-adjusted $p < 0.05$; Table F.2, Appendix F). These results indicate that, although this PER-AC-PPO $(1, 0)$ configuration explicitly models worker-related constraints in an online DRL framework, the proposed offline RL method can more effectively exploit static scheduling data and produce higher-quality schedules. Fig. 7 further illustrates the distribution of the best results achieved by each algorithm across the test instances. Compared with the baselines, MA-DQUAC yields a lower median and a narrower spread, suggesting more stable performance across the evaluated instances.

4.5. Ablation study

Eight settings were evaluated to isolate uncertainty weighting (UW) and the graph-attention encoder. The w/o UW variant omits UW from both actor and critic updates. Critic-only UW and Actor-only UW apply it only to the critic and actor updates, respectively, whereas MA-DQUAC applies it to both. In w/o HSG relations, relation-based message propagation is disabled while node features and the feasible-action mask are retained. In w/o multi-attention, the same neighborhoods are aggregated uniformly instead of using learned attention weights. The w/o WAB variant removes worker message attention but retains worker features. Single-head attention uses one rather than four heads in each operation, machine, and worker attention block. All settings use the Expert-Noisy dataset, the same training budget and test instances at 10 × 5 × 3 and 20 × 10 × 5; non-ablated components are unchanged. For Fig. 8(b), the relative Objective increase is $\Delta_{\text{abl}} = (\overline{C}^{\text{abl}}_{\max} - \overline{C}^{\text{full}}_{\max})/\overline{C}^{\text{full}}_{\max} \times 100\%$. Each mean is calculated over the same 100 test instances for a given scale and evaluation mode, with "full" denoting MA-DQUAC.

Table 5 and Fig. 8 show that MA-DQUAC achieved the lowest mean Objective in all four scale–evaluation combinations. Removing HSG relations, WAB, or multi-attention consistently increased the mean Objective. The largest increase occurred without multi-attention, although its relative magnitude declined from 55.6%–56.5% at 10 × 5 × 3 to 46.0%–49.8% at 20 × 10 × 5. Omitting UW also raised the mean Objective at both scales, by 14.9%–24.4% and 21.0%–21.2%, respectively. Paired Objective comparisons showed that MA-DQUAC outperformed all seven ablation variants at both scales under both evaluation modes (28/28 Holm-adjusted $p < 0.05$; Table F.3, Appendix F).

4.6. OOD calibration of quantile variance

The ablation study in Section 4.5 demonstrates the performance contribution of uncertainty weighting but does not directly verify whether the quantile variance in Eq. (17) reflects action-level distribution shift. Accordingly, quantile variance is treated as a task-specific OOD risk score rather than as sufficient evidence of low epistemic uncertainty or strong dataset support. We therefore conducted a state-matched calibration experiment. For each of the 10 × 5 × 3 and 20 × 10 × 5 scales, we generated 50 new scheduling instances and obtained one trajectory per instance using the Expert-Noisy strategy. Twelve non-terminal states were sampled from each trajectory, yielding 600 matched action pairs per scale. For each scale, all operation–machine–worker triplets observed across the offline training trajectories were collected in $\mathcal{A}_D$. At each sampled state s, the logged triplet in $\mathcal{A}_D$ served as the in-distribution (ID) action, while the OOD action was uniformly sampled from $\mathcal{A}(s) \setminus \mathcal{A}_D$, yielding a state-matched ID–OOD pair. No additional model fitting or parameter update was performed.

For each of five independently trained seeds, quantile variance was computed using Eq. (17). Paired differences were evaluated using a two-sided paired Wilcoxon signed-rank test and matched rank-biserial correlation, with Holm correction across the two scales. Receiver operating characteristic (ROC) curves were constructed independently for each seed, and the area under the ROC curve (AUROC) was computed together with the area under the precision–recall curve (AUPRC), using

Table 4
Results of baseline algorithms.

		10 × 5 × 3			15 × 10 × 5			20 × 10 × 5		
		Objective	Gap	Time	Objective	Gap	Time	Objective	Gap	Time
Greedy	DANIEL	482.64 ± 3.09	43.19 ± 0.92%	0.6621	367.39 ± 1.85	62.55 ± 0.82%	1.2669	552.62 ± 5.10	90.66 ± 1.76%	1.3777
	PER-AC-PPO (1,0)	413.97 ± 3.31	22.82 ± 0.98%	0.7271	257.26 ± 2.39	13.83 ± 1.06%	1.1023	294.18 ± 1.56	1.49 ± 0.54%	1.4792
	MA-DQUAC	**377.60** ± 2.76	**12.03** ± 0.82%	0.6890	**234.14** ± 1.13	**3.60** ± 0.50%	1.0553	**275.27** ± 1.16	**−5.03** ± 0.40%	1.4230
Sampling	DANIEL	560.73 ± 2.19	66.36 ± 0.65%	1.5125	453.23 ± 1.21	100.54 ± 0.54%	4.2351	873.96 ± 2.30	201.52 ± 0.79%	6.1982
	PER-AC-PPO (1,0)	389.51 ± 2.54	15.56 ± 0.75%	1.4298	237.76 ± 1.31	5.20 ± 0.58%	4.1085	274.95 ± 1.37	−5.14 ± 0.47%	6.5190
	MA-DQUAC	**327.99** ± 2.19	**−2.69** ± 0.65%	1.4050	**201.22** ± 0.91	**−10.97** ± 0.40%	4.0956	**241.56** ± 0.96	**−16.66** ± 0.33%	6.4337
PDRs	FIFO	679.48	101.59%	0.3411	591.93	161.90%	0.5223	752.86	159.74%	0.7781
	MOPNR	453.79	34.63%	0.3493	301.72	33.50%	0.5024	377.80	30.34%	0.9331
	MWKR	488.91	45.05%	0.3256	301.05	33.20%	0.4808	389.99	34.55%	0.9083
	SPT	990.99	194.01%	0.3012	892.11	294.72%	0.5254	1155.54	298.67%	0.8034
	GA	404.84 ± 1.37	20.11 ± 0.41%	104.3931	289.11 ± 0.92	27.92 ± 0.41%	156.0988	359.79 ± 0.99	24.13 ± 0.34%	359.7954
OR-Tools			337.06			226.01			289.85	

Note: Boldface indicates the best results for each evaluation mode within a given instance size.

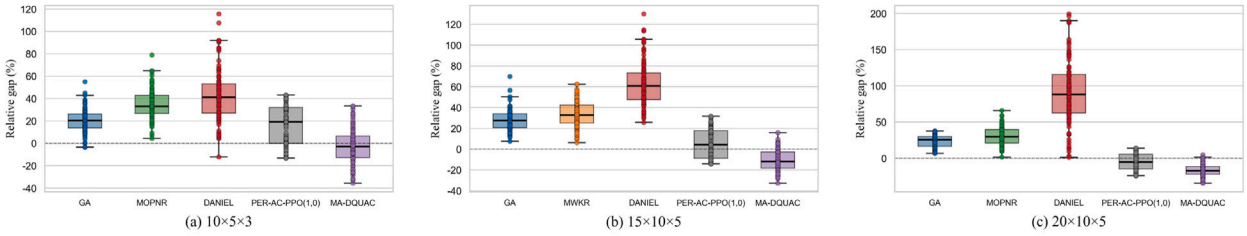


Fig. 7. Result distribution comparison of baseline algorithms.

Table 5
Results of uncertainty-weighting and structural/attention ablation studies.

Scale	Ablation variant	Greedy			Sampling		
		Objective	Gap	Time (s)	Objective	Gap	Time (s)
10 × 5 × 3	w/o UW	433.68 ± 3.55	28.67 ± 1.05%	0.6765	408.02 ± 2.87	21.05 ± 0.85%	1.3946
	Critic-only UW	424.06 ± 3.25	25.81 ± 0.97%	0.6803	387.27 ± 2.72	14.90 ± 0.81%	1.3980
	Actor-only UW	419.14 ± 3.19	24.35 ± 0.95%	0.6812	397.11 ± 2.59	17.82 ± 0.77%	1.3974
	w/o HSG relations	507.53 ± 3.36	50.58 ± 1.00%	0.6596	457.79 ± 2.79	35.82 ± 0.83%	1.3763
	w/o multi-attention	587.49 ± 2.88	74.30 ± 0.86%	0.6223	513.43 ± 2.42	52.33 ± 0.72%	1.3488
	w/o WAB	535.65 ± 3.02	58.92 ± 0.90%	0.6434	468.50 ± 2.45	39.00 ± 0.73%	1.3592
	Single-head attention	412.78 ± 2.72	22.46 ± 0.81%	0.6378	395.27 ± 2.09	17.27 ± 0.62%	1.3560
	MA-DQUAC	**377.60** ± 2.76	**12.03** ± 0.82%	0.6890	**327.99** ± 2.19	**−2.69** ± 0.65%	1.4050
20 × 10 × 5	w/o UW	333.15 ± 1.56	14.94 ± 0.54%	1.3895	292.72 ± 1.30	0.99 ± 0.45%	6.3430
	Critic-only UW	319.81 ± 1.46	10.34 ± 0.50%	1.3954	278.34 ± 1.21	−3.97 ± 0.42%	6.3496
	Actor-only UW	321.74 ± 1.45	11.00 ± 0.50%	1.3942	284.50 ± 1.19	−1.85 ± 0.41%	6.3672
	w/o HSG relations	338.61 ± 1.52	16.82 ± 0.52%	1.3702	299.10 ± 1.24	3.19 ± 0.43%	6.2151
	w/o multi-attention	412.44 ± 1.28	42.29 ± 0.44%	1.3112	352.59 ± 1.05	21.65 ± 0.36%	5.9719
	w/o WAB	386.92 ± 1.39	33.49 ± 0.48%	1.3557	329.04 ± 1.09	13.52 ± 0.38%	6.1744
	Single-head attention	311.96 ± 1.17	7.63 ± 0.40%	1.3510	287.32 ± 0.96	−0.87 ± 0.33%	6.1219
	MA-DQUAC	**275.27** ± 1.16	**−5.03** ± 0.40%	1.4230	**241.56** ± 0.96	**−16.66** ± 0.33%	6.4337

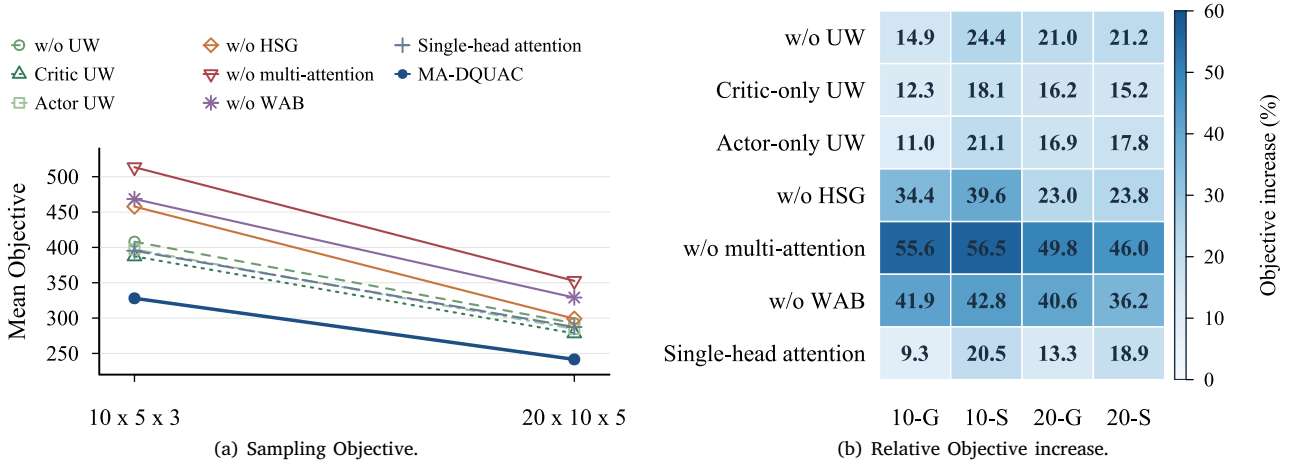
Note: Boldface indicates the best results for each evaluation mode within a given instance size.

quantile variance as the OOD score. All metrics were summarized as the mean and standard deviation across seeds.

As shown in Table 6 and Fig. 9, the median quantile variance of OOD actions was 2.07 ± 0.16 and 1.84 ± 0.12 times that of the matched ID actions at the two scales, respectively. All paired differences remained significant after Holm correction ($p_{\text{Holm}} < 0.001$). Quantile variance achieved AUROC values of 0.865 ± 0.024 and 0.850 ± 0.018, together with AUPRC values of 0.883 ± 0.024 and 0.849 ± 0.025. These results show that, in the deterministic scheduling setting considered here, quantile variance effectively distinguishes the ID and OOD actions defined above, supporting its use as a task-specific empirical proxy for action-level OOD uncertainty.

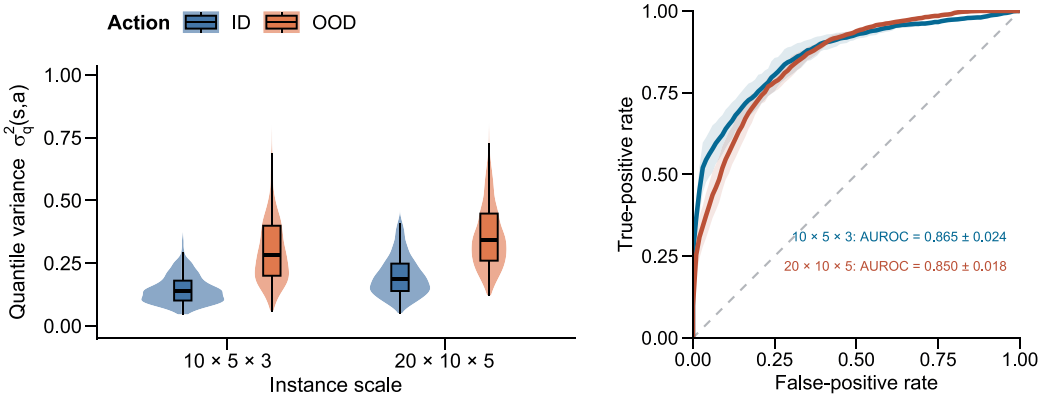
4.7. Sample efficiency analysis

To evaluate the sample efficiency of MA-DQUAC, experiments were conducted using different numbers of training samples, and the results are shown in Fig. 10. The algorithm achieves satisfactory scheduling results even in a low-sample environment with only 5 instances. Notably, as the number of training samples increases, the improvement in makespan is relatively small and tends to stabilize, indicating that MA-DQUAC has low data-scale dependence. This requirement is significantly lower than that of online RL (Zhang et al., 2020), which requires 1000 training instances. These results indicate that MA-DQUAC remained relatively stable across the tested reductions in training data.



(a) Sampling Objective. (b) Relative Objective increase.

Fig. 8. Ablation results across two instance scales. Panel (a) compares mean Objective under sampling evaluation. Panel (b) shows the percentage increase in mean Objective relative to MA-DQUAC under greedy (G) and sampling (S) evaluation.



(a) Quantile-variance distributions. (b) Receiver operating characteristic curves.

Fig. 9. Calibration of quantile variance for action-level OOD discrimination. In (a), violin and box plots show the quantile-variance distributions. In (b), solid lines and shaded bands show the mean ROC curves and ± one standard deviation across five seeds, respectively.

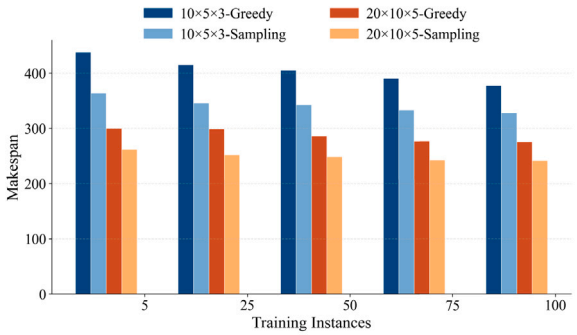


Fig. 10. Evaluation results for different numbers of training samples.

Table 6
Calibration results for action-level OOD discrimination using quantile variance.

Scale	ID median	OOD median	r_{rb}	AUROC	AUPRC
10 × 5 × 3	0.137 ±0.006	0.284 ±0.023	0.908 ±0.023	0.865 ±0.024	0.883 ±0.024
20 × 10 × 5	0.186 ±0.009	0.341 ±0.025	0.923 ±0.015	0.850 ±0.018	0.849 ±0.025

Note: r_{rb} denotes the matched rank-biserial correlation.

4.8. Generalization experiments

To evaluate the generalization performance of MA-DQUAC, this section directly applies the models trained on the Expert-Noisy dataset (with scales of 10 × 5 × 3 and 20 × 10 × 5, respectively) to unseen test instances beyond the training scale (30 × 10 × 8 and 40 × 10 × 8). As shown in Table 7, MA-DQUAC generalizes to larger and more complex scheduling instances than those used for training. In the 40 × 10 × 8 instances, the MA-DQUAC model trained on the 20 × 10 × 5 scale achieved a gap value of −24.83% under the sampling strategy, which is markedly better than the online RL baseline DANIEL (−4.29%), the meta-heuristic GA (4.63%), and the best-performing heuristic rule MOPNR (9.72%). MA-DQUAC achieved significantly lower Objective values in seven of the eight matched comparisons with DANIEL and all eight comparisons of the 20 × 10 × 5 model with GA and Best-PDR (Holm-adjusted $p < 0.05$). Mean differences and 95% confidence intervals are reported in Table F.4, Appendix F.

Furthermore, the model trained on the 20 × 10 × 5 scale achieved lower Objective values than the model trained on the 10 × 5 × 3

Table 7
Cross-scale generalization results on unseen test instances beyond the training scale.

Scale		Greedy				Sampling				GA	Best-PDR	OR-Tools
		DANIEL 10×5×3	DANIEL 20×10×5	MA-DQUAC 10×5×3	MA-DQUAC 20×10×5	DANIEL 10×5×3	DANIEL 20×10×5	MA-DQUAC 10×5×3	MA-DQUAC 20×10×5			
30 × 10 × 8	Objective	561.23 ± 3.16	528.76 ± 2.46	550.77 ± 2.12	**474.69** ± 1.99	509.37 ± 1.97	491.22 ± 1.65	475.17 ± 1.34	**422.34** ± 1.16	519.64 ± 1.50	557.11	480.41
	Gap	16.82% ± 0.66%	10.06% ± 0.51%	14.65% ± 0.44%	**−1.19%** ± 0.41%	6.03% ± 0.41%	2.25% ± 0.34%	−1.09% ± 0.28%	**−12.09%** ± 0.24%	8.17% ± 0.31%	15.97%	
	Time (s)	2.1937	2.2241	2.1612	2.1752	17.9729	18.3521	17.8780	18.0214	43.7133	1.2419	
40 × 10 × 8	Objective	765.12 ± 2.70	732.29 ± 2.53	691.04 ± 2.21	**598.96** ± 1.67	721.96 ± 1.79	688.85 ± 1.56	608.55 ± 1.35	**540.96** ± 1.10	752.98 ± 2.37	789.61	719.69
	Gap	6.31% ± 0.38%	1.75% ± 0.35%	−3.98% ± 0.31%	**−16.78%** ± 0.23%	0.32% ± 0.25%	−4.29% ± 0.22%	−15.44% ± 0.19%	**−24.83%** ± 0.15%	4.63% ± 0.33%	9.72%	
	Time (s)	3.1859	3.2976	3.1293	3.2636	29.1321	31.0728	29.2283	30.4367	68.1141	1.8922	

Note: Boldface indicates the best results for each evaluation mode within a given instance size.

Table 8
Inference scalability analysis under increasing job numbers.

Scale	Metric	MA-DQUAC		OR-Tools
		Greedy	Sampling	
50 × 10 × 8	Objective	691.20	**652.20**	1167.35
	Gap	−40.79%	**−44.13%**	
	Time (s)	4.97	45.38	
70 × 10 × 8	Objective	938.85	**887.75**	2606.85
	Gap	−63.99%	**−65.95%**	
	Time (s)	7.47	84.90	
100 × 10 × 8	Objective	1287.05	**1229.90**	3395.80
	Gap	−62.10%	**−63.78%**	
	Time (s)	11.37	174.69	
150 × 10 × 8	Objective	1862.55	**1806.30**	3521.90
	Gap	−47.12%	**−48.71%**	
	Time (s)	18.62	355.29	
200 × 10 × 8	Objective	2426.90	**2374.40**	3924.70
	Gap	−38.16%	**−39.50%**	
	Time (s)	26.13	637.77	
500 × 10 × 8	Objective	**5782.55**	–	9719.25
	Gap	**−40.50%**	–	
	Time (s)	89.88	–	
1000 × 10 × 8	Objective	**11 323.20**	–	–
	Gap	–	–	
	Time (s)	262.36	–	

Note: Boldface indicates the best results for each evaluation metric within a given instance size. "–" indicates that no valid result was obtained within 1800 s.

scale at both test scales under both inference strategies (all Holm-adjusted $p < 0.001$). This aligns with the expectation that larger training scales enable the model to capture feature representations that more closely approximate the distribution of larger test instances. On the tested larger-scale instances, MA-DQUAC obtains better solutions than the time-limited OR-Tools reference, indicating its ability to generate competitive schedules in larger solution spaces.

4.9. Scalability and limitations

The scalability of MA-DQUAC is evaluated by applying a fixed trained policy to instances with increasing numbers of jobs. Specifically, the model trained on the 20 × 10 × 5 Expert-Noisy dataset is directly tested on larger instances ranging from 50 × 10 × 8 to 1000 × 10 × 8, without retraining on these larger scales. This setting is used to analyze the inference-time growth and solution-quality variation after training. As shown in Fig. 11 and Table 8, when the instance size increases from 50 × 10 × 8 to 1000 × 10 × 8, the greedy inference time of MA-DQUAC increases from 4.97 s to 262.36 s, while still generating feasible schedules within a relatively short time. For

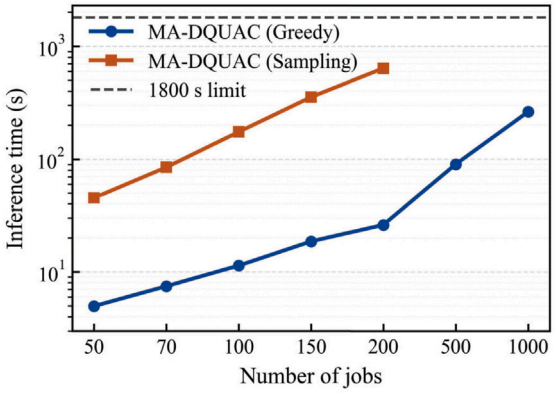


Fig. 11. Inference time growth of MA-DQUAC under increasing problem scales.

the 1000 × 10 × 8 instance, the actor evaluates at most 1000 × 10 × 8 = 80,000 current-step dense triplet positions in a batched forward pass, after which infeasible positions are masked out before policy normalization. The sampling strategy obtains better objective values from 50 × 10 × 8 to 200 × 10 × 8, but its inference time grows more rapidly. Therefore, greedy inference is more suitable for fast decision making on larger instances. In comparison, OR-Tools obtains worse objective values under the same time-limited setting and does not provide a valid result on the 1000 × 10 × 8 instance.

It should be noted that this experiment mainly reflects the inference-stage scalability of MA-DQUAC and should not be regarded as a full validation on all very large-scale DFJSP-W problems. As the number of jobs further increases, distribution shift and long-horizon decision errors may affect solution quality. In addition, instances with thousands of jobs would impose higher demands on offline data generation, state-graph construction, action-space handling, and model training. Therefore, extensions such as rolling-horizon scheduling, problem decomposition, hierarchical decision making, or distributed training are still needed for very large-scale industrial scenarios.

4.10. Case study analysis

Taking the precision reducer output flange machining scheduling problem of a mechanical component manufacturing enterprise in Beijing, China, as an example, the manufacturing process includes rough turning and datum surface machining, drilling and milling, high-frequency quenching and tempering, finish machining, and post-processing inspection, as shown in Fig. 12. Before machining, each operation requires worker preparation, such as material checking, fixture preparation, tool setting, positioning, and clamping. The case contains 8 flange workpieces, 5 sequential processes, 6 machines, and

Table 9
Processing times for different types of flanges/min.

Process	Machine	J_1	J_2	J_3	J_4	J_5	J_6	J_7	J_8
1	M_0	34	30	42	37	45	32	39	48
	M_1	38	35	47	41	50	36	44	54
2	M_1	20	18	24	22	27	19	23	30
	M_5	17	16	20	18	23	17	19	25
3	M_2	46	44	51	48	55	45	50	58
4	M_0	28	26	34	31	36	27	32	39
	M_3	21	19	25	23	27	20	24	29
5	M_4	12	11	14	13	15	11	13	16
	M_5	15	14	17	16	19	14	17	20

Table 10
Worker-preparation time data for flange machining operations/min.

Process	Machine	Type I			Type II			Type III		
		W_1	W_2	W_3	W_1	W_2	W_3	W_1	W_2	W_3
1	M_0	6	8	–	7	9	–	9	11	–
	M_1	8	9	–	9	10	–	11	12	–
2	M_1	–	6	8	–	7	9	–	9	11
	M_5	–	5	7	–	6	8	–	8	10
3	M_2	10	12	–	11	13	–	13	15	–
4	M_0	8	–	10	9	–	11	11	–	13
	M_3	6	–	8	7	–	9	9	–	11
5	M_4	–	5	6	–	6	7	–	8	9
	M_5	–	6	7	–	7	8	–	9	10

Note: "–" indicates that the corresponding worker is unavailable for the operation–machine setup assignment.

3 workers. The available machines and processing times are listed in Table 9, and the worker-preparation times are listed in Table 10. Since flange models with similar structures share the same preparation pattern, the data are grouped into three categories: Type I (J_1, J_2, and J_6), Type II (J_3, J_4, and J_7), and Type III (J_5 and J_8).

The scheduling results are shown in Fig. 13. OR-Tools obtains a schedule with a makespan of 559 min, but requires 1376.30 s of computation time. In contrast, MA-DQUAC generates a feasible schedule with a makespan of 567 min in only 1.79 s. The makespan gap relative to OR-Tools is 1.43%, while the inference speed is approximately 769 times faster. MA-DQUAC produced a feasible schedule with a makespan only 1.43% above the time-limited OR-Tools reference in this case. Although OR-Tools achieves a slightly smaller makespan, its long solving time limits its applicability in fast-response workshop scheduling. MA-DQUAC is therefore more suitable for practical decision-support scenarios where high-quality schedules must be generated repeatedly and efficiently under worker-machine collaboration constraints.

5. Conclusion

This study developed MA-DQUAC for learning worker-constrained flexible job shop schedules from fixed datasets. Across the evaluated instances, MA-DQUAC achieved lower makespans than the considered PDRs, GA, and adapted online and offline RL baselines. Policies trained on smaller instances transferred to larger unseen cases and remained relatively stable across the tested reductions in offline training data. On the larger benchmarks, MA-DQUAC also obtained lower makespans than the time-limited OR-Tools reference. In the industrial flange case, it generated a feasible schedule in 1.79 s with a makespan 1.43% above that obtained by OR-Tools. These findings support MA-DQUAC as a rapid decision-support method for the evaluated DFJSP-W settings. Future work will examine robustness to stochastic disruptions and improve scalability through offline-to-online adaptation, rolling-horizon scheduling, problem decomposition, and distributed training.

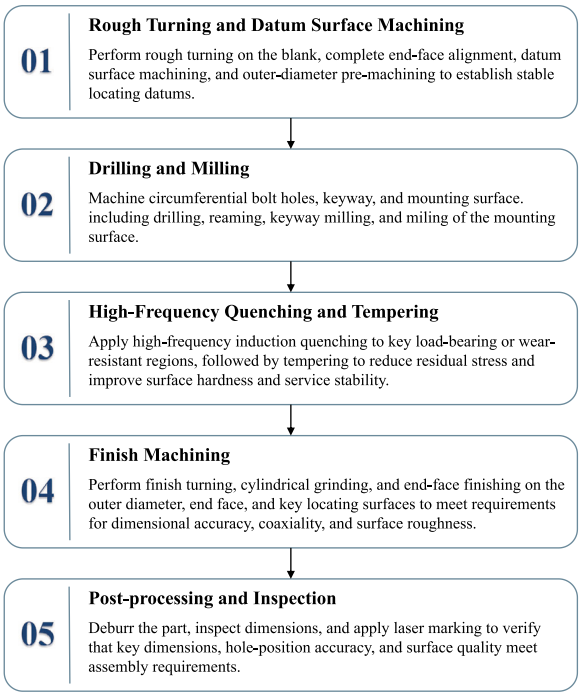
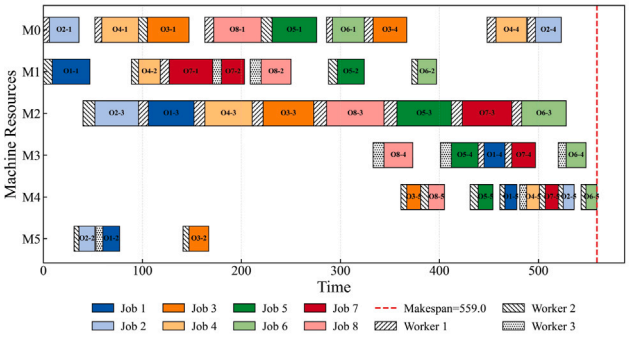
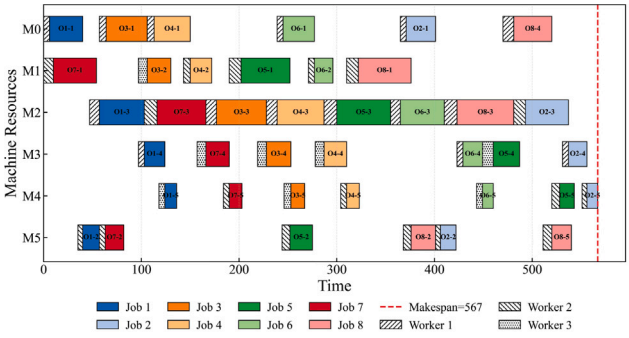


Fig. 12. Manufacturing process of the precision reducer output flange.



(a) Schedule obtained by OR-Tools.



(b) Schedule obtained by MA-DQUAC.

Fig. 13. Scheduling results for the precision reducer output flange case.

CRediT authorship contribution statement

Shiliang Shao: Writing – original draft, Methodology, Conceptualization. **Maolin Cai:** Visualization, Formal analysis. **Hongquan Qu:** Investigation, Data curation. **Qihui Yu:** Writing – review & editing,

Supervision. **Xiaomeng Tong:** Writing – review & editing, Validation, Resources, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research was supported by the National Key Research and Development Program of China (NO. 2024YFB3409700), the National Natural Science Foundation of China (NO. 52575046), and the Fundamental Research Funds for the Central Universities.

Appendix A. Mathematical model of the DFJSP-W

To formulate a well-structured mathematical programming model for the DFJSP-W, the relevant indices, sets, parameters, and decision variables are defined as follows.

(1) Indices and Sets

i: Indices for jobs, $i \in \{1, \dots, n\}$.

j: Indices for operations, $j \in \{1, \dots, n_i\}$.

k: Index for machines, $k \in \{1, \dots, K\}$.

l: Index for workers, $l \in \{1, \dots, L\}$.

$\mathcal{J}$: Set of all jobs, $\mathcal{J} = \{J_1, \dots, J_n\}$.

$\mathcal{O}$: Set of all operations, $\mathcal{O} = \{O_{ij} \mid i \in \{1, \dots, n\}, j \in \{1, \dots, n_i\}\}$.

$\mathcal{M}$: Set of all machines, $\mathcal{M} = \{M_1, \dots, M_K\}$.

$\mathcal{W}$: Set of all workers, $\mathcal{W} = \{W_1, \dots, W_L\}$.

$\mathcal{M}_{ij}$: Set of permissible machines for operation O_{ij}, $\mathcal{M}_{ij} \subseteq \mathcal{M}$.

$\mathcal{W}_{ijk}$: Set of permissible workers for operation O_{ij} on machine M_k, $\mathcal{W}_{ijk} \subseteq \mathcal{W}$.

V: A sufficiently large positive number.

(2) Parameters

n: Total number of jobs.

n_i: Total number of operations for job J_i.

p_{ijk}: Processing time of operation O_{ij} on machine M_k.

s_{ijkl}: Setup time incurred by worker W_l when preparing machine M_k for operation O_{ij}.

T_{ijkl}: Total machine occupancy time for operation O_{ij}, where $T_{ijkl} = s_{ijkl} + p_{ijk}$.

(3) Decision Variables

S_{ij}, C_{ij}: Setup start time and completion time of operation O_{ij}, respectively.

C_{max}: Maximum completion time of all jobs.

$x_{ijk} \in \{0, 1\}$: 1 if O_{ij} is assigned to machine M_k, 0 otherwise.

$z_{ijkl} \in \{0, 1\}$: 1 if worker W_l is assigned to machine M_k for operation O_{ij}, 0 otherwise.

$\delta^k_{iji'j'} \in \{0, 1\}$: 1 if O_{ij} precedes $O_{i'j'}$ on the same machine M_k, 0 otherwise.

$\zeta^l_{iji'j'} \in \{0, 1\}$: 1 if O_{ij} precedes $O_{i'j'}$ for the same worker W_l, 0 otherwise.

(4) Objective Function and Constraints

The objective is to minimize the makespan C_{max}:

$$\min C_{max} \tag{A.1}$$

Subject to:

$$\sum_{k \in \mathcal{M}_{ij}} x_{ijk} = 1, \quad \forall i, j \tag{A.2}$$

$$\sum_{l \in \mathcal{W}_{ijk}} z_{ijkl} = x_{ijk}, \quad \forall i, j, k \in \mathcal{M}_{ij} \tag{A.3}$$

$$S_{ij} + \sum_{k \in \mathcal{M}_{ij}} \sum_{l \in \mathcal{W}_{ijk}} z_{ijkl} \cdot T_{ijkl} = C_{ij}, \quad \forall i, j \tag{A.4}$$

$$C_{ij} \le S_{i(j+1)} + \sum_{k \in \mathcal{M}_{i(j+1)}} \sum_{l \in \mathcal{W}_{i(j+1)k}} z_{i(j+1)kl} \cdot s_{i(j+1)kl}, \quad \forall i, j \in \{1, \dots, n_i - 1\} \tag{A.5}$$

$$S_{ij} + \sum_{l \in \mathcal{W}_{ijk}} z_{ijkl} \cdot T_{ijkl} \le S_{i'j'} + V(1 - \delta^k_{iji'j'}) + V(2 - x_{ijk} - x_{i'j'k}), \quad \forall (i, j) \ne (i', j'), \forall k \in \mathcal{M}_{ij} \cap \mathcal{M}_{i'j'} \tag{A.6}$$

$$S_{i'j'} + \sum_{l \in \mathcal{W}_{i'j'k}} z_{i'j'kl} \cdot T_{i'j'kl} \le S_{ij} + V\delta^k_{iji'j'} + V(2 - x_{ijk} - x_{i'j'k}), \quad \forall (i, j) \ne (i', j'), \forall k \in \mathcal{M}_{ij} \cap \mathcal{M}_{i'j'} \tag{A.7}$$

$$S_{ij} + \sum_{k \in \mathcal{M}_{ij}} z_{ijkl} \cdot s_{ijkl} \le S_{i'j'} + V(1 - \zeta^l_{iji'j'}) + V(2 - \sum_{k \in \mathcal{M}_{ij}} z_{ijkl} - \sum_{k' \in \mathcal{M}_{i'j'}} z_{i'j'k'l}), \quad \forall (i, j) \ne (i', j'), \forall l \tag{A.8}$$

$$S_{i'j'} + \sum_{k' \in \mathcal{M}_{i'j'}} z_{i'j'k'l} \cdot s_{i'j'k'l} \le S_{ij} + V\zeta^l_{iji'j'} + V(2 - \sum_{k \in \mathcal{M}_{ij}} z_{ijkl} - \sum_{k' \in \mathcal{M}_{i'j'}} z_{i'j'k'l}), \quad \forall (i, j) \ne (i', j'), \forall l \tag{A.9}$$

$$C_{ij} \le C_{max}, \quad \forall i, j \tag{A.10}$$

$$S_{ij}, C_{ij}, C_{max} \ge 0; \quad x_{ijk}, z_{ijkl}, \delta^k_{iji'j'}, \zeta^l_{iji'j'} \in \{0, 1\} \tag{A.11}$$

Eq. (A.1) defines the objective to minimize the overall makespan. Eq. (A.2) ensures that each operation is assigned to exactly one compatible machine. Eq. (A.3) guarantees that exactly one skilled worker is selected for the assigned machine. Eq. (A.4) defines the completion time as the sum of setup start time and total machine occupancy duration (setup plus processing). Eq. (A.5) ensures that the processing of each subsequent operation begins after its predecessor is completed while allowing anticipatory setup. Eqs. (A.6) and (A.7) are machine-level non-overlap constraints, ensuring a machine is assigned to only one operation at a time. Eqs. (A.8) and (A.9) prevent overlapping setup tasks assigned to the same worker; the worker is released immediately after setup. Eq. (A.10) defines the makespan as the maximum completion time across all jobs. Finally, Eq. (A.11) specifies the permissible values for all variables.

Appendix B. Definitions of state features

Here, we describe in detail the state features used for handling the DFJSP-W. The job features ($H_O \in \mathbb{R}^{10}$), machine features ($H_M \in \mathbb{R}^8$), and operation–machine pair features ($H_{OM} \in \mathbb{R}^8$) follow the definitions proposed by Wang et al. (2023) and are thus not repeated. The additional worker features ($H_W \in \mathbb{R}^6$) and machine–worker pair features ($H_{MW} \in \mathbb{R}^4$) are introduced as follows:

1. **Features of Workers:** Each worker $W_l \in \mathcal{W}$ owns a feature vector with six elements representing their current status and capability.
   - Number of available candidate operations: the count of candidate operations for which worker W_l is currently eligible to perform setup.
   - Number of compatible unscheduled operations: the total number of remaining operations in the workshop for which worker W_l is skilled to perform setup.

- Minimum setup time: the shortest setup time for worker W_l across all compatible operation–machine setup assignments.
- Average setup time: the mean setup time of all operations that worker W_l can set up.
- Free Time: the moment when worker W_l is available to start a new setup task.
- Working Tag: the value is 0 if worker W_l is idle at the current schedule time, otherwise 1.

2. **Machine–Worker Pair Features:** For each compatible operation–machine–worker assignment, four machine–worker pair features are recorded to characterize the resource coupling efficiency:

- Relative global setup time: the ratio of the current setup time to the maximum setup time among all remaining unscheduled operations.
- Relative worker-specific remaining setup time: the ratio of the current setup time to the maximum remaining setup time specifically for worker W_l.
- Relative worker-specific candidate setup time: the ratio of the current setup time to the maximum setup time among all current candidate operations for worker W_l.
- Relative worker efficiency: the ratio of the current setup time to the average setup time of all feasible workers for the current operation.

Appendix C. Computation of the estimated completion-time indicator

To make the reward calculation reproducible, Algorithm 1 summarizes the update of the estimated completion-time indicator $C_{\text{est}}(s_t)$ after each scheduling decision.

Algorithm 1: Computation of the Estimated Completion-Time Indicator and Reward

Input: Current state (s_t), selected action ($a_t = (O_{ij}, M_k, W_l)$), current completion-time estimates ($\hat{C}$), job available time (A_i), machine available time (A_k), worker available time (A_l), processing time (p_{ijk}), setup time (s_{ijkl})

Output: Updated estimated completion-time indicator ($C_{\text{est}}(s_{t+1})$), reward (r_t)

$C_{\text{old}} \leftarrow C_{\text{est}}(s_t)$;
$S_{ij} \leftarrow \max\{A_k, A_l, A_i - s_{ijkl}\}$;
$\hat{C}_{ij}^{new} \leftarrow S_{ij} + s_{ijkl} + p_{ijk}$;
$A_l \leftarrow S_{ij} + s_{ijkl}$;
$A_k \leftarrow \hat{C}_{ij}^{new}$;
$A_i \leftarrow \hat{C}_{ij}^{new}$;
$\Delta \leftarrow \hat{C}_{ij}^{new} - \hat{C}_{ij}^{old}$;
for $q \leftarrow j$ **to** n_i **do**
    $\hat{C}_{iq} \leftarrow \hat{C}_{iq} + \Delta$;
$C_{\text{est}}(s_{t+1}) \leftarrow \max_i \hat{C}_{in_i}$;
$r_t \leftarrow C_{\text{old}} - C_{\text{est}}(s_{t+1})$;

Appendix D. Hyperparameter tuning curves

Fig. D.1 presents the hyperparameter tuning curves. For each subplot, only the corresponding hyperparameter is varied, while the other settings are fixed to the default values or to the best values identified in previous tuning steps.

Appendix E. Implementation, adaptation, and fairness protocol for baseline methods

This appendix consolidates the implementation, task-specific adaptation, and fairness protocol for all comparison baselines. The baselines are organized as PDRs, OR-Tools, GA, and RL methods. For methods that do not involve policy learning, state, action, reward, training, hyperparameter tuning, and random-seed information are reported only where applicable.

E.1. Priority dispatching rules

Four representative PDRs are selected as comparison baselines. These rules are extended to simultaneously consider machine availability and worker constraints during the scheduling process to adapt to the DFJSP-W environment, as described below:

(1) First In First Out (FIFO): selects the earliest ready candidate operation and the earliest available compatible machine and worker.

(2) Most Operations Remaining (MOPNR): selects the candidate operation belonging to the job with the maximum number of remaining operations.

(3) Shortest Processing Time (SPT): selects the compatible operation–machine–worker assignment that has the minimum processing time.

(4) Most Work Remaining (MWKR): selects the candidate operation from the job with the highest remaining workload, calculated as the total average processing time of all its unfinished operations.

E.2. OR-tools

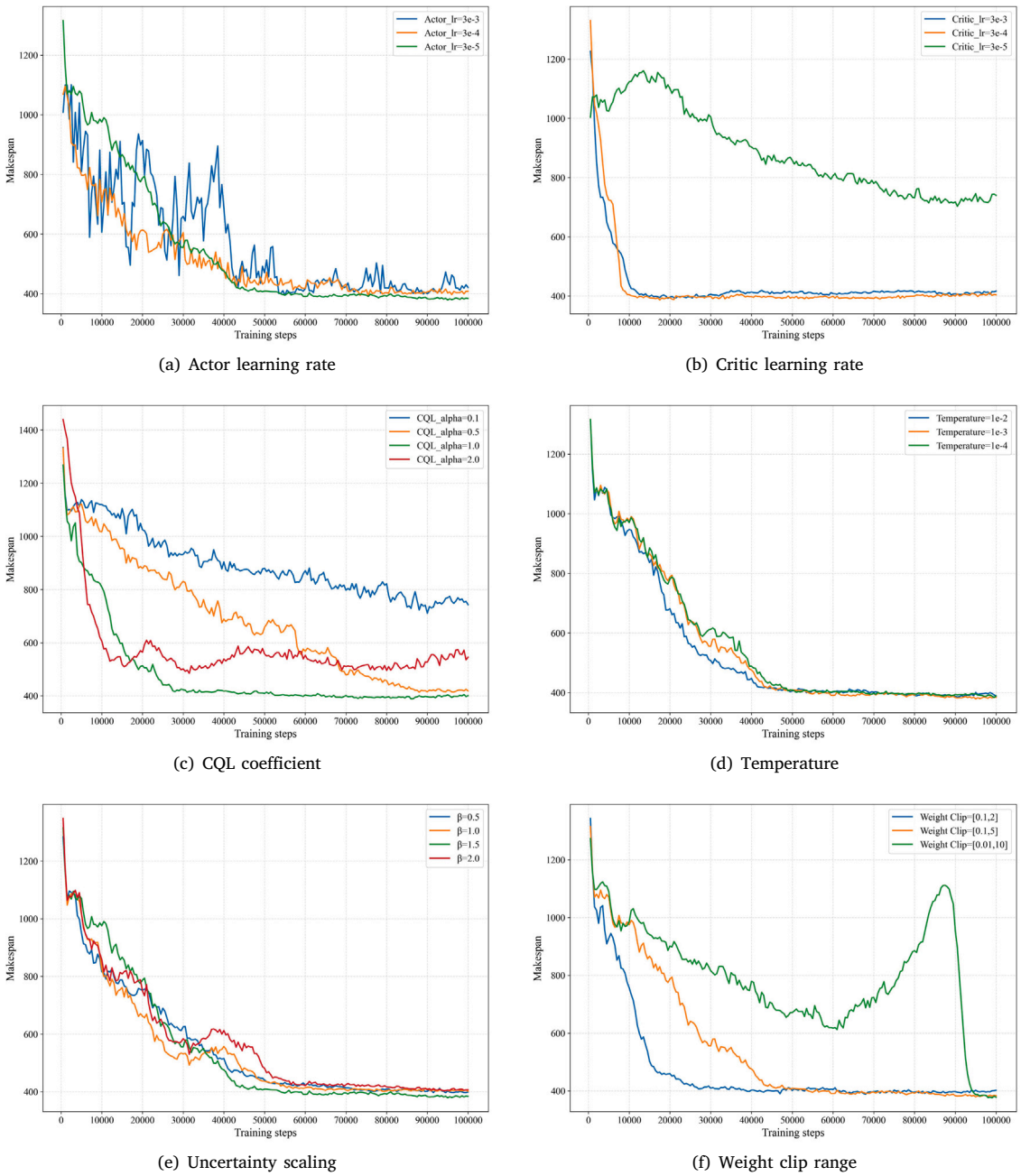
OR-Tools CP-SAT was used as a time-limited reference solver. For each operation, binary variables select exactly one feasible operation–machine–worker assignment. Interval and no-overlap constraints enforce operation precedence, machine non-overlap during setup and processing, and worker non-overlap during setup, while the objective minimizes the makespan. Each test instance was solved for at most 1800 s using eight parallel search workers with the solver random seed fixed at 0; if optimality was not proven, the best feasible solution returned within the time limit was recorded. No additional parameter tuning was used.

E.3. Genetic algorithm

The GA uses a three-part chromosome comprising the operation sequence, machine assignment, and worker assignment. During decoding, operation precedence and machine–worker compatibility constraints are enforced, and the makespan is used as the fitness value. The parent and offspring populations are merged at each generation, and the individuals with the smallest makespan values are retained. Hyperparameters were selected using a limited sequential grid search on the held-out $10 \times 5 \times 3$ validation set, in which one parameter was varied at a time and the setting with the smallest mean validation makespan was selected without using the test set. The final population size, number of generations, crossover probability, and mutation probability were 200, 50, 0.8, and 0.05, respectively. Each test instance was independently solved using random seeds $\{0, 1, 2, 3, 4\}$, and the objective and runtime were averaged over the five runs.

E.4. Reinforcement-learning methods

CDQAC, DANIEL, and PER-AC-PPO are implemented in the same DFJSP-W environment, with identical transitions, compatibility constraints, and evaluation protocols. Each adaptation adds only the task interfaces required by DFJSP-W while retaining the baseline's original representation and learning mechanism.



(a) Actor learning rate

(b) Critic learning rate

(c) CQL coefficient

(d) Temperature

(e) Uncertainty scaling

(f) Weight clip range

Fig. D.1. Validation makespan curves under different tuned hyperparameter values.

E.4.1. CDQAC adaptation

State representation. CDQAC retains its original operation, machine, and operation–machine pair features, denoted by $H_O \in \mathbb{R}^{10}$, $H_M \in \mathbb{R}^8$, and $H_{OM} \in \mathbb{R}^8$, respectively. These features are processed by its dual-attention encoder. For DFJSP-W, we add the worker features $H_W \in \mathbb{R}^6$ and machine–worker pair features $H_{MW} \in \mathbb{R}^4$ defined in Appendix B. The parameter-shared worker MLP has two fully connected layers with dimensions $6 \rightarrow 32 \rightarrow 8$. An ELU activation follows the 32-unit hidden layer, and the 8-dimensional output layer is linear. Mean pooling provides a global worker embedding to the policy and twin quantile critics.

Action space, transition, and reward. CDQAC's operation–machine action is extended to the triplet $a_t = (O_{ij}, M_k, W_l)$, which is scored from the entity embeddings and corresponding pair features. The common feasibility mask removes triplets that violate candidate-operation, operation–machine, or machine–worker compatibility requirements. Resource availability determines the earliest feasible setup start time. Each feasible action follows the transition in Section 3.1 and receives the dense makespan reward in Eq. (6).

Offline dataset and preserved mechanisms. CDQAC is trained on the same static transition dataset as MA-DQUAC for each instance scale and behavior-policy setting, without additional trajectories or online interactions. It retains the dual-attention encoder, twin dueling quantile critics, CQL regularization, and delayed policy updates. It does not use MA-DQUAC's uncertainty weighting or additional attention modules.

E.4.2. DANIEL adaptation

State representation. DANIEL retains its original operation–machine representation. The features $H_O \in \mathbb{R}^{10}$, $H_M \in \mathbb{R}^8$, and $H_{OM} \in \mathbb{R}^8$ are processed by alternating operation-message and machine-message

Table F.1
Paired differences in Objective between MA-DQUAC and CDQAC for the Table 3 conditions.

Scale	Strategy	Training dataset	Mean difference	95% CI	Holm-adjusted p
$10 \times 5 \times 3$	Greedy	Mix-PDRs	−22.78	[−43.81, −1.75]	0.204
		GA	−7.41	[−20.66, 5.84]	0.784
		Expert	−9.58	[−18.61, −0.55]	0.204
		Expert-Noisy	−6.57	[−17.99, 4.85]	0.784
		Random	−183.92	[−209.03, −158.81]	6.24×10^{-25}
	Sampling	Mix-PDRs	−56.51	[−69.36, −43.66]	9.06×10^{-13}
		GA	3.87	[−2.03, 9.77]	0.784
		Expert	−22.53	[−27.31, −17.75]	4.37×10^{-14}
		Expert-Noisy	−19.29	[−24.74, −13.84]	3.50×10^{-9}
		Random	−149.31	[−163.73, −134.89]	4.87×10^{-36}
$15 \times 10 \times 5$	Greedy	Mix-PDRs	−28.85	[−40.42, −17.28]	3.07×10^{-5}
		GA	−39.12	[−54.13, −24.11]	1.33×10^{-5}
		Expert	−19.15	[−26.01, −12.29]	2.96×10^{-6}
		Expert-Noisy	−2.98	[−7.53, 1.57]	0.784
		Random	−174.04	[−200.45, −147.63]	4.91×10^{-22}
	Sampling	Mix-PDRs	−65.40	[−70.81, −59.99]	1.38×10^{-41}
		GA	−14.96	[−21.79, −8.13]	2.71×10^{-4}
		Expert	−19.31	[−22.15, −16.47]	7.18×10^{-23}
		Expert-Noisy	−11.92	[−13.73, −10.11]	4.70×10^{-22}
		Random	−165.23	[−181.76, −148.70]	7.88×10^{-35}
$20 \times 10 \times 5$	Greedy	Mix-PDRs	−36.20	[−50.83, −21.57]	3.22×10^{-5}
		GA	−81.90	[−96.76, −67.04]	1.70×10^{-17}
		Expert	−12.71	[−20.29, −5.13]	0.00867
		Expert-Noisy	−23.26	[−28.32, −18.20]	1.40×10^{-13}
		Random	−278.16	[−319.30, −237.02]	1.05×10^{-22}
	Sampling	Mix-PDRs	−42.33	[−48.29, −36.37]	4.94×10^{-24}
		GA	−54.42	[−60.81, −48.03]	1.51×10^{-29}
		Expert	−33.70	[−37.65, −29.75]	1.27×10^{-29}
		Expert-Noisy	−24.29	[−26.39, −22.19]	5.14×10^{-40}
		Random	−186.47	[−206.29, −166.65]	8.57×10^{-33}

attention blocks. The added worker branch uses $H_W \in \mathbb{R}^6$ and $H_{MW} \in \mathbb{R}^4$ from Appendix B. Each worker is encoded by the same parameter-shared two-layer MLP ($6 \to 32 \to 8$), with ELU after the hidden layer and a linear output layer. Mean pooling provides the 8-dimensional global worker embedding to the actor and critic. No worker-specific attention, message-passing, or competition module is added.

Action space and transition. DANIEL's action (O_{ij}, M_k) is extended to $a_t = (O_{ij}, M_k, W_l)$. A fully connected network scores each triplet from its entity embeddings, pair features, and pooled global state. The common feasibility mask is applied before softmax normalization. The selected triplet updates the operation status and the availability times of both resources; the worker is released after setup, whereas the machine is released after processing.

Reward and preserved mechanisms. DANIEL uses the dense makespan reward in Eq. (6). Its original attention modules, machine-competition mechanism, PPO optimizer, and two decoding strategies are retained. The adaptation does not include MA-DQUAC's offline-learning components.

E.4.3. PER-AC-PPO adaptation

State representation. PER-AC-PPO retains its heterogeneous operation–machine–worker graph and NHGNN backbone. The normalized tensors in Section 3.1 and Appendix B are mapped to the corresponding node and compatibility-edge types, while triplet-related pair attributes are supplied to the actor. Because fatigue and operating cost are not part of the present state, no artificial features are introduced for them.

Action space and transition. The original triplet action $a_t = (O_{ij}, M_k, W_l)$ is retained. Candidate triplets combine each unfinished job's next operation with compatible machines and workers. The common feasibility mask enforces candidate-operation and resource-compatibility requirements, while the transition applies the same precedence and resource-capacity rules used by the other RL baselines.

Reward and preserved mechanisms. Hu et al. (2025) independently train policies for predefined reward-weight vectors, including the makespan endpoint $(\omega_1, \omega_2) = (1, 0)$ (ω_1: makespan; ω_2: total cost). We retrain PER-AC-PPO from scratch at this original endpoint using the common makespan reward in Eq. (6). The reported baseline therefore represents its *adapted makespan-oriented configuration*, rather than the complete Pareto framework. The NHGNN, triplet decision, Actor–Critic PPO, prioritized replay, and adaptive clipping from 0.20 to 0.05 are retained.

E.4.4. Training budget, hyperparameter selection, and random seeds

CDQAC is trained offline with Adam for 100,000 gradient steps using batches of 256. DANIEL and PER-AC-PPO are trained for at most 4000 policy-update cycles with 20 parallel environments, producing up to 80,000 complete trajectories per seed. Both online baselines are validated every 10 cycles on the same held-out $10 \times 5 \times 3$ set. The checkpoint with the smallest mean validation makespan is used for testing. Their interaction budget, validation frequency, checkpoint rule, and stopping limit are identical.

Hyperparameters are selected without using the test set. CDQAC adopts the published configuration of van Remmerden et al. (2025a), without an additional search. We use $\eta = 4$, $\alpha_{\mathrm{CQL}} = 0.05$, $N = 64$, $\gamma = 1.0$, $\rho = 0.005$, and an entropy coefficient of 0.005. The policy and quantile-critic learning rates are 2×10^{-5} and 2×10^{-4}, respectively. The dual-attention encoder has two layers and four heads. The policy and quantile critic each have two hidden layers of 64 units.

DANIEL retains its reference configuration: a learning rate of 3×10^{-4}, two dual-attention layers, four heads per block, and two 64-unit hidden layers in both the actor and critic. We use $\gamma = 1.0$, a PPO clipping coefficient of 0.20, and an entropy coefficient of 0.01. Each update uses four PPO epochs and a minibatch size of 1024. The worker MLP is fixed to the architecture reported above, without an additional

Table F.2
Paired Objective differences between MA-DQUAC and the baselines in Table 4.

Scale	Strategy	Baseline	Mean difference	95% CI	Holm-adjusted p
$10\times5\times3$	Greedy	DANIEL	−105.04	[−124.49, −85.59]	2.42×10^{-17}
		PER-AC-PPO (1, 0)	−36.37	[−57.88, −14.86]	0.00113
		GA	−27.24	[−39.41, −15.07]	9.09×10^{-5}
		FIFO	−301.88	[−318.58, −285.18]	4.27×10^{-57}
		MOPNR	−76.19	[−89.98, −62.40]	8.02×10^{-18}
		MWKR	−111.31	[−124.98, −97.64]	1.85×10^{-28}
		SPT	−613.39	[−636.53, −590.25]	1.14×10^{-72}
	Sampling	DANIEL	−232.74	[−247.85, −217.63]	7.25×10^{-51}
		PER-AC-PPO (1, 0)	−61.52	[−78.33, −44.70]	4.45×10^{-10}
		GA	−76.85	[−86.56, −67.14]	1.22×10^{-27}
		FIFO	−351.49	[−366.13, −336.85]	1.39×10^{-68}
		MOPNR	−125.80	[−137.66, −113.94]	3.82×10^{-37}
		MWKR	−160.92	[−172.61, −149.23]	1.40×10^{-46}
		SPT	−663.00	[−685.02, −640.98]	5.63×10^{-78}
$15\times10\times5$	Greedy	DANIEL	−133.25	[−144.04, −122.46]	1.41×10^{-42}
		PER-AC-PPO (1, 0)	−23.12	[−35.89, −10.35]	0.00103
		GA	−54.97	[−61.15, −48.79]	2.98×10^{-31}
		FIFO	−357.79	[−367.86, −347.72]	7.29×10^{-85}
		MOPNR	−67.58	[−74.69, −60.47]	1.94×10^{-33}
		MWKR	−66.91	[−73.89, −59.93]	1.05×10^{-33}
		SPT	−657.97	[−678.93, −637.01]	1.05×10^{-79}
	Sampling	DANIEL	−252.01	[−259.35, −244.67]	2.03×10^{-83}
		PER-AC-PPO (1, 0)	−36.54	[−45.07, −28.02]	1.37×10^{-12}
		GA	−87.89	[−93.56, −82.22]	4.42×10^{-51}
		FIFO	−390.71	[−399.94, −381.48]	2.96×10^{-92}
		MOPNR	−100.50	[−107.18, −93.82]	5.55×10^{-50}
		MWKR	−99.83	[−105.78, −93.88]	3.61×10^{-54}
		SPT	−690.89	[−711.16, −670.62]	3.84×10^{-83}
$20\times10\times5$	Greedy	DANIEL	−277.35	[−303.46, −251.24]	3.56×10^{-37}
		PER-AC-PPO (1, 0)	−18.91	[−27.35, −10.47]	9.09×10^{-5}
		GA	−84.52	[−91.62, −77.42]	3.15×10^{-41}
		FIFO	−477.59	[−487.80, −467.38]	1.70×10^{-96}
		MOPNR	−102.53	[−110.38, −94.68]	1.21×10^{-44}
		MWKR	−114.72	[−121.82, −107.62]	1.06×10^{-52}
		SPT	−880.27	[−898.89, −861.65]	6.09×10^{-97}
	Sampling	DANIEL	−632.40	[−645.09, −619.71]	3.84×10^{-99}
		PER-AC-PPO (1, 0)	−33.39	[−42.31, −24.47]	2.35×10^{-10}
		GA	−118.23	[−124.83, −111.63]	8.96×10^{-57}
		FIFO	−511.30	[−521.32, −501.28]	3.70×10^{-100}
		MOPNR	−136.24	[−143.82, −128.66]	7.28×10^{-57}
		MWKR	−148.43	[−155.08, −141.78]	1.28×10^{-65}
		SPT	−913.98	[−933.04, −894.92]	1.53×10^{-97}

search. For PER-AC-PPO, we repeat the validation grid reported by Hu et al. (2025), covering learning rates $\{10^{-2}, 10^{-3}, 10^{-4}\}$, MLP depths $\{1, 2, 3, 4\}$, and hidden dimensions $\{32, 64, 128\}$. The selected configuration uses 10^{-4}, three MLP layers, and 64 hidden units. Except for the matched interaction budget and validation schedule above, the fixed settings follow Hu et al. (2025): Adam optimization, $\gamma = 1.0$, an entropy coefficient of 0.01, zero dropout, an update interval of $E_u = 10$, and 1024 experience samples in the prioritized replay buffer. The adaptive clipping coefficient decreases from 0.20 to 0.05, as described in Section E.4.3.

All three baselines use random seeds $\{0, 1, 2, 3, 4\}$ for training and evaluation. The CDQAC seed controls initialization, offline minibatch sampling, and stochastic action sampling. For the online baselines, it controls instance sampling, initialization, and stochastic action sampling. Table 3 reports the five-seed mean for CDQAC, and Table 4 reports the corresponding means for DANIEL and PER-AC-PPO (1, 0).

Appendix F. Statistical comparisons for Tables 3–5 and 7

For methods evaluated with multiple seeds, Objective values were averaged across the five seeds for each test instance before paired comparisons. For each scale, inference strategy, and training dataset in Table 3, the Objective values of MA-DQUAC and CDQAC were paired by test instance ($n = 100$). We calculated the instance-level difference as MA-DQUAC minus CDQAC; negative values favor MA-DQUAC. Pointwise 95% confidence intervals for the mean difference and two-sided paired t-tests were based on these 100 differences. The 30 p-values were adjusted together using the Holm procedure. Confidence intervals are pointwise and are not adjusted for multiple comparisons (see Table F.1).

Table F.2 reports Objective differences between MA-DQUAC and the baselines in Table 4 across 100 matched test instances per condition. Differences are defined as MA-DQUAC minus the baseline, so negative values favor MA-DQUAC. Pointwise 95% confidence intervals and two-sided paired t-tests were calculated from instance-level differences; p-values were Holm-adjusted across the 42 comparisons.

Table F.3 compares MA-DQUAC with the seven ablation variants in Table 5 on the same 100 test instances per condition. Differences are defined as MA-DQUAC minus the ablation variant; negative values favor MA-DQUAC. Pointwise 95% confidence intervals and two-sided paired t-tests were calculated from instance-level differences, and the 28 p-values were adjusted together using the Holm procedure.

Table F.3
Paired Objective differences between MA-DQUAC and the ablation variants in Table 5.

Scale	Strategy	Ablation variant	Mean difference	95% CI	Holm-adjusted p
$10 \times 5 \times 3$	Greedy	w/o UW	−56.08	[−78.18, −33.98]	8.56×10^{-6}
		Critic-only UW	−46.46	[−67.34, −25.58]	7.76×10^{-5}
		Actor-only UW	−41.54	[−62.82, −20.26]	3.85×10^{-4}
		w/o HSG relations	−129.93	[−150.39, −109.47]	4.38×10^{-21}
		w/o multi-attention	−209.89	[−230.57, −189.21]	2.02×10^{-35}
		w/o WAB	−158.05	[−177.45, −138.65]	3.22×10^{-28}
		Single-head attention	−35.18	[−54.49, −15.87]	4.74×10^{-4}
	Sampling	w/o UW	−80.03	[−96.90, −63.16]	2.11×10^{-14}
		Critic-only UW	−59.28	[−76.39, −42.17]	2.82×10^{-9}
		Actor-only UW	−69.12	[−86.48, −51.76]	2.37×10^{-11}
		w/o HSG relations	−129.80	[−146.37, −113.23]	4.84×10^{-27}
		w/o multi-attention	−185.44	[−202.07, −168.81]	1.02×10^{-38}
		w/o WAB	−140.51	[−157.36, −123.66]	6.26×10^{-29}
		Single-head attention	−67.28	[−82.28, −52.28]	2.17×10^{-13}
$20 \times 10 \times 5$	Greedy	w/o UW	−57.88	[−67.77, −47.99]	4.60×10^{-19}
		Critic-only UW	−44.54	[−53.42, −35.66]	1.52×10^{-15}
		Actor-only UW	−46.47	[−55.65, −37.29]	1.08×10^{-15}
		w/o HSG relations	−63.34	[−71.74, −54.94]	6.54×10^{-26}
		w/o multi-attention	−137.17	[−146.66, −127.68]	2.57×10^{-48}
		w/o WAB	−111.65	[−121.52, −101.78]	3.09×10^{-39}
		Single-head attention	−36.69	[−44.98, −28.40]	3.52×10^{-13}
	Sampling	w/o UW	−51.16	[−59.72, −42.60]	1.53×10^{-19}
		Critic-only UW	−36.78	[−44.77, −28.79]	7.52×10^{-14}
		Actor-only UW	−42.94	[−50.12, −35.76]	1.53×10^{-19}
		w/o HSG relations	−57.54	[−64.38, −50.70]	3.46×10^{-29}
		w/o multi-attention	−111.03	[−118.06, −104.00]	9.92×10^{-52}
		w/o WAB	−87.48	[−94.33, −80.63]	1.15×10^{-43}
		Single-head attention	−45.76	[−52.40, −39.12]	2.69×10^{-23}

Table F.4
Paired Objective differences for the cross-scale generalization results in Table 7.

Test scale	Strategy	Comparison	Mean difference	95% CI	Holm-adjusted p
$30 \times 10 \times 8$	Greedy	MA-DQUAC$_{10}$ vs DANIEL$_{10}$	−10.46	[−29.44, 8.52]	0.277
		MA-DQUAC$_{20}$ vs DANIEL$_{20}$	−54.07	[−69.32, −38.82]	1.04×10^{-9}
		MA-DQUAC$_{20}$ vs GA	−44.95	[−58.29, −31.61]	4.10×10^{-9}
		MA-DQUAC$_{20}$ vs Best-PDR	−82.42	[−96.91, −67.93]	1.06×10^{-18}
		MA-DQUAC$_{20}$ vs MA-DQUAC$_{10}$	−76.08	[−86.59, −65.57]	4.72×10^{-25}
	Sampling	MA-DQUAC$_{10}$ vs DANIEL$_{10}$	−34.20	[−45.71, −22.69]	1.05×10^{-7}
		MA-DQUAC$_{20}$ vs DANIEL$_{20}$	−68.88	[−79.09, −58.67]	4.22×10^{-23}
		MA-DQUAC$_{20}$ vs GA	−97.30	[−107.55, −87.05]	1.63×10^{-33}
		MA-DQUAC$_{20}$ vs Best-PDR	−134.77	[−147.25, −122.29]	7.90×10^{-38}
		MA-DQUAC$_{20}$ vs MA-DQUAC$_{10}$	−52.83	[−56.96, −48.70]	5.98×10^{-44}
$40 \times 10 \times 8$	Greedy	MA-DQUAC$_{10}$ vs DANIEL$_{10}$	−74.08	[−91.79, −56.37]	2.74×10^{-12}
		MA-DQUAC$_{20}$ vs DANIEL$_{20}$	−133.33	[−149.38, −117.28]	3.46×10^{-29}
		MA-DQUAC$_{20}$ vs GA	−154.02	[−168.88, −139.16]	2.07×10^{-36}
		MA-DQUAC$_{20}$ vs Best-PDR	−190.65	[−205.78, −175.52]	2.13×10^{-43}
		MA-DQUAC$_{20}$ vs MA-DQUAC$_{10}$	−92.08	[−101.54, −82.62]	2.66×10^{-34}
	Sampling	MA-DQUAC$_{10}$ vs DANIEL$_{10}$	−113.41	[−124.72, −102.10]	2.72×10^{-35}
		MA-DQUAC$_{20}$ vs DANIEL$_{20}$	−147.89	[−156.86, −138.92]	1.26×10^{-53}
		MA-DQUAC$_{20}$ vs GA	−212.02	[−225.16, −198.88]	8.48×10^{-53}
		MA-DQUAC$_{20}$ vs Best-PDR	−248.65	[−263.65, −233.65]	8.01×10^{-54}
		MA-DQUAC$_{20}$ vs MA-DQUAC$_{10}$	−67.59	[−71.64, −63.54]	4.89×10^{-54}

Table F.4 reports paired Objective comparisons for Table 7, using 100 matched test instances per condition. Subscripts 10 and 20 denote training scales of $10 \times 5 \times 3$ and $20 \times 10 \times 5$, respectively. Comparisons comprise MA-DQUAC versus DANIEL at the same training scale, MA-DQUAC$_{20}$ versus GA and Best-PDR, and MA-DQUAC$_{20}$ versus MA-DQUAC$_{10}$. Differences are the first model minus the comparator, so negative values favor the first model. Pointwise 95% confidence intervals and two-sided paired t-tests were calculated from these differences, with Holm adjustment across all 20 comparisons.

Data availability

Research Link Provided

DFJSP-W Datasets for Human--Machine Collaborative Flexible Job Shop Scheduling (Original data) (GitHub)

References

An, J., Cai, H., Zhao, Y., Gui, X., He, X., & Jin, X. (2025). JSHM: A dynamic flexible job-shop scheduling method with human-machine collaboration. *Neurocomputing*, Article 132213.

Aribi, D., Driss, O. B., & El Haouzi, H. B. (2023). A real-time double flexible job shop scheduling problem under industry 5.0. In *2023 IEEE afro-mediterranean conference on artificial intelligence* (pp. 1–7). IEEE.

Bai, Y., Gao, Y., Wan, R., Zhang, S., & Song, R. (2025). A review of reinforcement learning in financial applications. *Annual Review of Statistics and its Application*, *12*(1), 209–232.

Brandimarte, P. (1993). Routing and scheduling in a flexible job shop by tabu search. *Annals of Operations Research*, *41*(3), 157–183.

Chang, X., Jia, X., & Ren, J. (2025). A reinforcement learning enhanced memetic algorithm for multi-objective flexible job shop scheduling toward industry 5.0. *International Journal of Production Research*, *63*(1), 119–147.

Chen, H., Ihlow, J., & Lehmann, C. (1999). A genetic algorithm for flexible job-shop scheduling. In *Proceedings 1999 IEEE international conference on robotics and automation (cat. no. 99CH36288C): Vol. 2*, (pp. 1120–1125). Ieee.

Christodoulou, P. (2019). Soft actor-critic for discrete action settings. http://dx.doi.org/10.48550/arXiv.1910.07207, arXiv preprint arXiv:1910.07207.

Dabney, W., Rowland, M., Bellemare, M., & Munos, R. (2018). Distributional reinforcement learning with quantile regression. In *Proceedings of the AAAI conference on artificial intelligence: Vol. 32, No. 1*, (pp. 2892–2901). http://dx.doi.org/10.1609/aaai.v32i1.11791.

Dauzère-Pérès, S., Ding, J., Shen, L., & Tamssaouet, K. (2024). The flexible job shop scheduling problem: A review. *European Journal of Operational Research*, *314*(2), 409–432.

Defersha, F. M., Obimuyiwa, D., & Yimer, A. D. (2022). Mathematical model and simulated annealing algorithm for setup operator constrained flexible job shop scheduling problem. *Computers & Industrial Engineering*, *171*, Article 108487.

Destouet, C., Tlahig, H., Bettayeb, B., & Mazari, B. (2023). Flexible job shop scheduling problem under industry 5.0: A survey on human reintegration, environmental consideration and resilience improvement. *Journal of Manufacturing Systems*, *67*, 155–173.

Fujimoto, S., Meger, D., & Precup, D. (2019). Off-policy deep reinforcement learning without exploration. In *International conference on machine learning* (pp. 2052–2062). PMLR.

Ghasemi, P., Greenberg, M., Southern, D. A., Li, B., White, J. A., & Lee, J. (2025). Personalized decision making for coronary artery disease treatment using offline reinforcement learning. *Npj Digital Medicine*, *8*(1), 99.

Hu, Y., Zhang, Z., Chen, J., Tang, D., & Cai, Q. (2025). Graph-based deep reinforcement learning for dynamic scheduling of flexible job-shop considering worker fatigue and multi-skill factors. *Applied Soft Computing*, Article 113712.

Janner, M., Li, Q., & Levine, S. (2021). Offline reinforcement learning as one big sequence modeling problem. *Advances in Neural Information Processing Systems*, *34*, 1273–1286.

Kumar, A., Zhou, A., Tucker, G., & Levine, S. (2020). Conservative q-learning for offline reinforcement learning. *Advances in Neural Information Processing Systems*, *33*, 1179–1191.

Lan, L., & Berkhout, J. (2025). PyJobShop: Solving scheduling problems with constraint programming in Python. http://dx.doi.org/10.48550/arXiv.2502.13483, arXiv preprint arXiv:2502.13483.

Leng, J., Sha, W., Wang, B., Zheng, P., Zhuang, C., Liu, Q., Wuest, T., Mourtzis, D., & Wang, L. (2022). Industry 5.0: Prospect and retrospect. *Journal of Manufacturing Systems*, *65*, 279–295.

Li, X., & Gao, L. (2016). An effective hybrid genetic algorithm and tabu search for flexible job shop scheduling problem. *International Journal of Production Economics*, *174*, 93–110.

Li, Z., Tong, X., Shi, P., Cai, M., & Han, F. (2025). Classification of non-standard mechanical parts with graph convolutional networks. *Engineering Applications of Artificial Intelligence*, *153*, Article 110879.

Li, X., Zhao, Q., Tang, H., Guo, X., Zhuang, M., Li, Y., & Wang, X. V. (2024). Flexible job shop composite dispatching rule mining approach based on an improved genetic programming algorithm. *Tsinghua Science and Technology*, *29*(5), 1390–1408.

Liu, M., Lv, J., Du, S., Deng, Y., Shen, X., & Zhou, Y. (2024). Multi-resource constrained flexible job shop scheduling problem with fixture-pallet combinatorial optimisation. *Computers & Industrial Engineering*, *188*, Article 109903.

Moon, S., Lee, S., & Park, K. J. (2024). Learning-enabled flexible job-shop scheduling for scalable smart manufacturing. *Journal of Manufacturing Systems*, *77*, 356–367.

Naderi, B., & Roshanaei, V. (2022). Critical-path-search logic-based benders decomposition approaches for flexible job shop scheduling. *INFORMS Journal on Optimization*, *4*(1), 1–28.

Nair, A., Gupta, A., Dalal, M., & Levine, S. (2020). Awac: Accelerating online reinforcement learning with offline datasets. http://dx.doi.org/10.48550/arXiv.2006.09359, arXiv preprint arXiv:2006.09359.

Pang, J., Mei, Y., & Zhang, M. (2025). A multiform many-objective genetic programming method for dynamic flexible job shop scheduling. In *Proceedings of the genetic and evolutionary computation conference* (pp. 267–276).

Prudencio, R. F., Maximo, M. R., & Colombini, E. L. (2023). A survey on offline reinforcement learning: Taxonomy, review, and open problems. *IEEE Transactions on Neural Networks and Learning Systems*, *35*(8), 10237–10257.

Qu, H., Shao, S., Xu, Y., Cai, M., Shi, Y., & Tong, X. (2026). A global feedback learning-based memetic algorithm for energy-aware scheduling in collaborative heterogeneous flexible job shops. *Swarm and Evolutionary Computation*, *104*, Article 102392.

Qu, H., Tong, X., Cai, M., Shi, Y., & Lan, X. (2024). Energy-saving scheduling strategy for variable-speed flexible job-shop problem considering operation-dependent energy consumption. *Expert Systems with Applications*, *256*, Article 124952.

Qu, H., Tong, X., Shao, S., Cai, M., Shi, Y., Ma, Y., & Ning, F. (2026). Collaborative evolution guided by large language models for heterogeneous flexible job-shop scheduling with cooperative units. *Chinese Journal of Mechanical Engineering*, Article 100374.

Sels, V., Gheysen, N., & Vanhoucke, M. (2012). A comparison of priority rules for the job shop scheduling problem under different flow time-and tardiness-related objective functions. *International Journal of Production Research*, *50*(15), 4255–4270.

Sobeyko, O., & Mönch, L. (2016). Heuristic approaches for scheduling jobs in large-scale flexible job shops. *Computers & Operations Research*, *68*, 97–109.

Song, W., Chen, X., Li, Q., & Cao, Z. (2022). Flexible job-shop scheduling via graph neural network and deep reinforcement learning. *IEEE Transactions on Industrial Informatics*, *19*(2), 1600–1610.

Su, R., Fu, H., Zhou, S., & Fu, Y. (2025). Integrating offline pre-training with online fine-tuning: A reinforcement learning approach for robot social navigation. In *2025 IEEE/RSJ international conference on intelligent robots and systems* (pp. 18660–18667). IEEE.

Thenarasu, M., Rameshkumar, K., Rousseau, J., & Anbuudayasankar, S. (2022). Development and analysis of priority decision rules using MCDM approach for a flexible job shop scheduling: A simulation study. *Simulation Modelling Practice and Theory*, *114*, Article 102416.

van Remmerden, J., Bukhsh, Z., & Zhang, Y. (2025a). Generalizing beyond suboptimality: Offline reinforcement learning learns effective scheduling through random data. http://dx.doi.org/10.48550/arXiv.2509.10303, arXiv preprint arXiv:2509.10303.

van Remmerden, J., Bukhsh, Z., & Zhang, Y. (2025b). Offline reinforcement learning for learning to dispatch for job shop scheduling. *Machine Learning*, *114*(8), 191.

Wang, H., Hao, S., Dong, H., Zhang, S., Bao, Y., Yang, Z., & Wu, Y. (2025). Offline reinforcement learning for llm multi-step reasoning. In *Findings of the association for computational linguistics: ACL 2025* (pp. 8881–8893).

Wang, Z., Schaul, T., Hessel, M., Hasselt, H., Lanctot, M., & Freitas, N. (2016). Dueling network architectures for deep reinforcement learning. In *International conference on machine learning* (pp. 1995–2003). PMLR.

Wang, R., Wang, G., Sun, J., Deng, F., & Chen, J. (2023). Flexible job shop scheduling via dual attention network-based reinforcement learning. *IEEE Transactions on Neural Networks and Learning Systems*, *35*(3), 3091–3102.

Wang, Y., Wu, J., He, H., Wei, Z., & Sun, F. (2025). Data-driven energy management for electric vehicles using offline reinforcement learning. *Nature Communications*, *16*(1), 2835.

Wu, Y., Zhai, S., Srivastava, N., Susskind, J., Zhang, J., Salakhutdinov, R., & Goh, H. (2021). Uncertainty weighted actor-critic for offline reinforcement learning. http://dx.doi.org/10.48550/arXiv.2105.08140, arXiv preprint arXiv:2105.08140.

Xie, J., Gao, L., Peng, K., Li, X., & Li, H. (2019). Review on flexible job shop scheduling. *IET Collaborative Intelligent Manufacturing*, *1*(3), 67–77.

Xiong, F., & Liu, H. (2025). Logic-based benders decomposition methods for the distributed flexible job shop scheduling problem. *European Journal of Operational Research*.

Xu, X., Lu, Y., Vogel-Heuser, B., & Wang, L. (2021). Industry 4.0 and industry 5.0—Inception, conception and perception. *Journal of Manufacturing Systems*, *61*, 530–535.

Xu, M., Mei, Y., Zhang, F., Ong, Y. S., & Zhang, M. (2025). Pareto set learning through genetic programming for multi-objective dynamic scheduling. *IEEE Transactions on Evolutionary Computation*.

Yang, R., Wang, D., Li, M., Cui, C., & Qiao, J. (2025). Enhancing offline reinforcement learning for wastewater treatment via transition filter and prioritized approximation loss. *Neurocomputing*, *636*, Article 129977.

Yang, J., Wang, H., & Xiong, J. (2025). Evolutionary strategies with dual graph reinforcement learning for flexible job shop scheduling problem. In *2025 IEEE congress on evolutionary computation* (pp. 1–8). IEEE.

Yuan, M., Zheng, L., Huang, H., Zhou, K., Pei, F., & Gu, W. (2025). Research on flexible job shop scheduling problem with AGV using double DQN. *Journal of Intelligent Manufacturing*, *36*(1), 509–535.

Yuan, S., Zhu, X., Gao, J., & Zhang, R. (2025). Flexible job shop scheduling with peak power consumption constraints: mathematical models and an improved memetic algorithm. *Journal of the Operational Research Society*, 1–19.

Zhang, R., Dai, B., Li, L., & Schuurmans, D. (2020). Gendice: Generalized offline estimation of stationary values. http://dx.doi.org/10.48550/arXiv.2002.09072, arXiv preprint arXiv:2002.09072.

Zhang, J., Ding, G., Zou, Y., Qin, S., & Fu, J. (2019). Review of job shop scheduling research and its new perspectives under industry 4.0. *Journal of Intelligent Manufacturing*, *30*(4), 1809–1830.

Zhang, G., Gao, L., & Shi, Y. (2011). An effective genetic algorithm for the flexible job-shop scheduling problem. *Expert Systems with Applications*, *38*(4), 3563–3573.

Zhang, C., Song, W., Cao, Z., Zhang, J., Tan, P. S., & Chi, X. (2020). Learning to dispatch for job shop scheduling via deep reinforcement learning. *Advances in Neural Information Processing Systems*, *33*, 1621–1632.

Zhang, X., & Zhu, G. Y. (2025). A literature review of reinforcement learning methods applied to job-shop scheduling problems. *Computers & Operations Research*, *175*, Article 106929.

Zhong, R., Feng, Y., Zhang, A., Li, C., Jin, K., Li, Z., & Tan, J. (2025). Solving flexible job-shop problem considering skilled workers via multi-domain graph attention network. *International Journal of Production Research*, 1–22.

Zhou, G., Ke, L., Srinivasa, S., Gupta, A., Rajeswaran, A., & Kumar, V. (2022). Real world offline reinforcement learning with realistic data source. http://dx.doi.org/10.48550/arXiv.2210.06479, arXiv preprint arXiv:2210.06479.